

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent
Kind Code
Date of Patent
Inventor(s)

12417170
B2
September 16, 2025
Tseng; Po-Hao et al.

Computing system and method of operation thereof

Abstract

A 3D search engine receives searches for application to word lines of a nonvolatile memory array. The engine uses two word lines per bit of information of the searches and two memory devices per bit of stored feature to search against, optionally enabling don't care and/or wildcard encoding. The engine uses respective bit lines of the nonvolatile memory array as respective matching lines for searching. Respective memory strings (e.g., NAND memory strings) of the nonvolatile memory array are usable to store respective data words, e.g., corresponding to features to search for. Respective pluralities of the memory strings are coupled in parallel to respective shared bit lines. Various encodings of features and searches enable exact, approximate, and range matching. The engine has applicability to comparing and sorting, in addition to searching in application areas such as artificial intelligence (AI) and big data.

Inventors: Tseng; Po-Hao (Taichung, TW), Lee; Ming-Hsiu (Hsinchu, TW), Bo; Tian-Cih (Zhubei, TW)

Applicant: MACRONIX INTERNATIONAL CO., LTD. (Hsinchu, TW)

Family ID: 1000008819255

Assignee: MACRONIX INTERNATIONAL CO., LTD. (Hsinchu, TW)

Appl. No.: 18/195540

Filed: May 10, 2023

Prior Publication Data

Document Identifier	Publication Date
US 20240378144 A1	Nov. 14, 2024

Publication Classification

Int. Cl.: G06F12/00 (20060101); G06F12/02 (20060101)

U.S. Cl.:

CPC **G06F12/0246** (20130101);

Field of Classification Search

CPC: G11C (11/4096); G11C (11/5671); G11C (15/00); G11C (15/04); G11C (16/02); G11C (16/3459); G11C (2211/5641); G11C (2211/5642); G11C (2213/71); G11C (2213/75); G11C (7/1039); G11C (7/1042); G11C (7/1072); G11C (7/12); G11C (8/08); G11C (11/5628); G11C (16/24); G11C (16/3427); G11C (11/005); G11C (11/4063); G11C (11/419); G11C (11/5621); G11C (14/0063); G11C (16/00); G11C (16/04); G11C (16/30); G11C (7/1006); G11C (7/1069); G11C (7/1096); G06F (12/0246); G06F (15/7821); G06F (3/0604); G06F (3/0658); G06F (3/0659); H01L (2225/06562); H01L (25/0657); H01L (2225/06506); H01L (2225/0651); H01L (2225/06513); H01L (2225/06517); H01L (2225/06541); H01L (25/18); H04L (2012/5685); H10B (43/10); H10B (43/27); H10B (69/00)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
4219829	12/1979	Dorda et al.	N/A	N/A
4987090	12/1990	Hsu et al.	N/A	N/A
5029130	12/1990	Yeh	N/A	N/A
5586073	12/1995	Hiura et al.	N/A	N/A
5963803	12/1998	Dawson et al.	N/A	N/A
6034882	12/1999	Johnson et al.	N/A	N/A
6107882	12/1999	Gabara et al.	N/A	N/A
6313486	12/2000	Kencke et al.	N/A	N/A
6385097	12/2001	Liao et al.	N/A	N/A
6486027	12/2001	Noble et al.	N/A	N/A
6576385	12/2002	Bode et al.	N/A	N/A
6593624	12/2002	Walker	N/A	N/A
6829598	12/2003	Milev	N/A	N/A
6856542	12/2004	Roy et al.	N/A	N/A
6906940	12/2004	Lue	N/A	N/A
6960499	12/2004	Nandakumar et al.	N/A	N/A
7050318	12/2005	Argyres	N/A	N/A
7081377	12/2005	Cleeves	N/A	N/A
7089218	12/2005	Visel	N/A	N/A
7114026	12/2005	Khanna	N/A	N/A
7129538	12/2005	Lee et al.	N/A	N/A
7177169	12/2006	Scheuerlein	N/A	N/A
7324381	12/2007	Gallo et al.	N/A	N/A
7368358	12/2007	Ouyang et al.	N/A	N/A
7436723	12/2007	Rinerson et al.	N/A	N/A
7593908	12/2008	Abdulkader et al.	N/A	N/A
7646041	12/2009	Chae et al.	N/A	N/A
7747668	12/2009	Nomura et al.	N/A	N/A
7907432	12/2010	Deshpande et al.	N/A	N/A

7948024	12/2010	Kim et al.	N/A	N/A
8045355	12/2010	Ueda	N/A	N/A
8154128	12/2011	Lung	N/A	N/A
8203187	12/2011	Lung et al.	N/A	N/A
8275728	12/2011	Pino	N/A	N/A
8331149	12/2011	Choi et al.	N/A	N/A
8432719	12/2012	Lue	N/A	N/A
8564045	12/2012	Liu	N/A	N/A
8589320	12/2012	Breitwisch et al.	N/A	N/A
8630114	12/2013	Lue	N/A	N/A
8725670	12/2013	Visel	N/A	N/A
8837189	12/2013	Roohparvar	365/49.1	G11C 16/0483
8860124	12/2013	Lue et al.	N/A	N/A
9064903	12/2014	Mitchell et al.	N/A	N/A
9111617	12/2014	Shim et al.	N/A	N/A
9147468	12/2014	Lue	N/A	N/A
9177966	12/2014	Rabkin et al.	N/A	N/A
9213936	12/2014	Visel	N/A	N/A
9379129	12/2015	Lue et al.	N/A	N/A
9384835	12/2015	Arsovski et al.	N/A	N/A
9391084	12/2015	Lue	N/A	N/A
9397110	12/2015	Lue	N/A	N/A
9401371	12/2015	Lee et al.	N/A	N/A
9430735	12/2015	Vali et al.	N/A	N/A
9431099	12/2015	Lee et al.	N/A	N/A
9520485	12/2015	Lue	N/A	N/A
9524980	12/2015	Lue	N/A	N/A
9530499	12/2015	Bando et al.	N/A	N/A
9535831	12/2016	Jayasena et al.	N/A	N/A
9536969	12/2016	Yang et al.	N/A	N/A
9589982	12/2016	Cheng et al.	N/A	N/A
9698156	12/2016	Lue	N/A	N/A
9698185	12/2016	Chen et al.	N/A	N/A
9710747	12/2016	Kang et al.	N/A	N/A
9747230	12/2016	Han et al.	N/A	N/A
9754953	12/2016	Tang et al.	N/A	N/A
9767028	12/2016	Cheng et al.	N/A	N/A
9898207	12/2017	Kim et al.	N/A	N/A
9910605	12/2017	Jayasena et al.	N/A	N/A
9922716	12/2017	Hsiung et al.	N/A	N/A
9978454	12/2017	Jung	N/A	N/A
9983829	12/2017	Ravimohan et al.	N/A	N/A
9991007	12/2017	Lee et al.	N/A	N/A
10037167	12/2017	Kwon et al.	N/A	N/A
10043819	12/2017	Lai et al.	N/A	N/A
10056149	12/2017	Yamada et al.	N/A	N/A
10073733	12/2017	Jain et al.	N/A	N/A
10157012	12/2017	Kelner et al.	N/A	N/A
10175667	12/2018	Bang et al.	N/A	N/A
10211218	12/2018	Lue	N/A	N/A
10242737	12/2018	Lin et al.	N/A	N/A

10283205	12/2018	Malshe et al.	N/A	N/A
10381376	12/2018	Nishikawa et al.	N/A	N/A
10403637	12/2018	Lue	N/A	N/A
10528643	12/2019	Choi et al.	N/A	N/A
10534840	12/2019	Petti	N/A	N/A
10540591	12/2019	Gao et al.	N/A	N/A
10552759	12/2019	Rich	N/A	N/A
10565494	12/2019	Henry et al.	N/A	N/A
10635398	12/2019	Lin et al.	N/A	N/A
10643713	12/2019	Louie et al.	N/A	N/A
10719296	12/2019	Lee et al.	N/A	N/A
10777566	12/2019	Lue	N/A	N/A
10783963	12/2019	Hung et al.	N/A	N/A
10790023	12/2019	Harari	N/A	N/A
10790828	12/2019	Gunter et al.	N/A	N/A
10825510	12/2019	Jaiswal et al.	N/A	N/A
10860682	12/2019	Knag et al.	N/A	N/A
10910393	12/2020	Lai et al.	N/A	N/A
10942673	12/2020	Shafiee Ardestani et al.	N/A	N/A
10957392	12/2020	Lee et al.	N/A	N/A
11069704	12/2020	Lai et al.	N/A	N/A
11127108	12/2020	Sharma et al.	N/A	N/A
11328775	12/2021	Tseng et al.	N/A	N/A
11410028	12/2021	Crill et al.	N/A	N/A
11443407	12/2021	Sharma et al.	N/A	N/A
11737274	12/2022	Shen	N/A	N/A
11892902	12/2023	Liu	N/A	N/A
11916011	12/2023	Lai et al.	N/A	N/A
11934480	12/2023	Lue et al.	N/A	N/A
11955178	12/2023	Kawasumi	N/A	N/A
12086410	12/2023	Mathuriya et al.	N/A	N/A
2001/0055838	12/2000	Walker et al.	N/A	N/A
2002/0028541	12/2001	Lee et al.	N/A	N/A
2003/0122181	12/2002	Wu	N/A	N/A
2005/0088878	12/2004	Eshel	N/A	N/A
2005/0219887	12/2004	Hsu et al.	N/A	N/A
2005/0280061	12/2004	Lee	N/A	N/A
2005/0287793	12/2004	Blanchet et al.	N/A	N/A
2007/0158736	12/2006	Arai et al.	N/A	N/A
2008/0101109	12/2007	Haring-Bolivar et al.	N/A	N/A
2008/0117678	12/2007	Shieh et al.	N/A	N/A
2008/0298110	12/2007	Wickeraad et al.	N/A	N/A
2009/0097321	12/2008	Kim et al.	N/A	N/A
2009/0184360	12/2008	Jin et al.	N/A	N/A
2009/0190404	12/2008	Roohparvar	N/A	N/A
2010/0172189	12/2009	Itagaki et al.	N/A	N/A
2010/0182828	12/2009	Shima et al.	N/A	N/A
2010/0202208	12/2009	Endo et al.	N/A	N/A
2010/0270593	12/2009	Lung et al.	N/A	N/A
2011/0018051	12/2010	Kim et al.	N/A	N/A

2011/0063915	12/2010	Tanaka et al.	N/A	N/A
2011/0106742	12/2010	Pino	N/A	N/A
2011/0128791	12/2010	Chang et al.	N/A	N/A
2011/0140070	12/2010	Kim	N/A	N/A
2011/0194357	12/2010	Han et al.	N/A	N/A
2011/0286258	12/2010	Chen et al.	N/A	N/A
2011/0297912	12/2010	Samachisa et al.	N/A	N/A
2012/0007167	12/2011	Hung et al.	N/A	N/A
2012/0044742	12/2011	Narayanan	N/A	N/A
2012/0112264	12/2011	Lee et al.	N/A	N/A
2012/0182801	12/2011	Lue	N/A	N/A
2012/0235111	12/2011	Osano et al.	N/A	N/A
2012/0254087	12/2011	Visel	N/A	N/A
2013/0070528	12/2012	Maeda	N/A	N/A
2013/0075684	12/2012	Kinoshita et al.	N/A	N/A
2013/0119455	12/2012	Chen et al.	N/A	N/A
2014/0043898	12/2013	Kuo et al.	N/A	N/A
2014/0063949	12/2013	Tokiwa	N/A	N/A
2014/0119127	12/2013	Lung et al.	N/A	N/A
2014/0149773	12/2013	Huang et al.	N/A	N/A
2014/0153310	12/2013	Sekar et al.	N/A	N/A
2014/0268996	12/2013	Park	N/A	N/A
2014/0330762	12/2013	Visel	N/A	N/A
2015/0008500	12/2014	Fukumoto et al.	N/A	N/A
2015/0170001	12/2014	Rabinovich et al.	N/A	N/A
2015/0171106	12/2014	Suh	N/A	N/A
2015/0179661	12/2014	Huo et al.	N/A	N/A
2015/0199126	12/2014	Jayasena et al.	N/A	N/A
2015/0331817	12/2014	Han et al.	N/A	N/A
2015/0340369	12/2014	Lue	N/A	N/A
2016/0043100	12/2015	Lee et al.	N/A	N/A
2016/0141299	12/2015	Hong	N/A	N/A
2016/0141337	12/2015	Shimabukuro et al.	N/A	N/A
2016/0181315	12/2015	Lee et al.	N/A	N/A
2016/0232973	12/2015	Jung	N/A	N/A
2016/0247579	12/2015	Ueda et al.	N/A	N/A
2016/0308114	12/2015	Kim et al.	N/A	N/A
2016/0329341	12/2015	Shimabukuro et al.	N/A	N/A
2016/0336064	12/2015	Seo et al.	N/A	N/A
2016/0342892	12/2015	Ross	N/A	N/A
2016/0342893	12/2015	Ross et al.	N/A	N/A
2016/0343421	12/2015	Pyo	N/A	N/A
2016/0358661	12/2015	Vali et al.	N/A	N/A
2016/0379115	12/2015	Burger et al.	N/A	N/A
2017/0003889	12/2016	Kim et al.	N/A	N/A
2017/0025421	12/2016	Sakakibara et al.	N/A	N/A
2017/0084748	12/2016	Yang	N/A	N/A
2017/0092370	12/2016	Harari	N/A	N/A
2017/0103316	12/2016	Ross et al.	N/A	N/A
2017/0123987	12/2016	Cheng et al.	N/A	N/A
2017/0148517	12/2016	Harari	N/A	N/A

2017/0160955	12/2016	Jayasena et al.	N/A	N/A
2017/0169885	12/2016	Tang et al.	N/A	N/A
2017/0169887	12/2016	Widjaja	N/A	N/A
2017/0243879	12/2016	Yu et al.	N/A	N/A
2017/0263623	12/2016	Zhang et al.	N/A	N/A
2017/0270405	12/2016	Kurokawa	N/A	N/A
2017/0287928	12/2016	Kanamori et al.	N/A	N/A
2017/0309634	12/2016	Noguchi et al.	N/A	N/A
2017/0316833	12/2016	Ihm et al.	N/A	N/A
2017/0317096	12/2016	Shin et al.	N/A	N/A
2017/0337466	12/2016	Bayat et al.	N/A	N/A
2018/0113649	12/2017	Shafiee Ardestani et al.	N/A	N/A
2018/0121790	12/2017	Kim et al.	N/A	N/A
2018/0129424	12/2017	Confalonieri et al.	N/A	N/A
2018/0129936	12/2017	Young et al.	N/A	N/A
2018/0144240	12/2017	Garbin et al.	N/A	N/A
2018/0157488	12/2017	Shu et al.	N/A	N/A
2018/0173420	12/2017	Li et al.	N/A	N/A
2018/0182776	12/2017	Kim	N/A	N/A
2018/0189640	12/2017	Henry et al.	N/A	N/A
2018/0240522	12/2017	Jung	N/A	N/A
2018/0246783	12/2017	Avraham et al.	N/A	N/A
2018/0247195	12/2017	Kumar et al.	N/A	N/A
2018/0286874	12/2017	Kim et al.	N/A	N/A
2018/0321942	12/2017	Yu et al.	N/A	N/A
2018/0342299	12/2017	Yamada et al.	N/A	N/A
2018/0350823	12/2017	Or-Bach et al.	N/A	N/A
2019/0019538	12/2018	Li et al.	N/A	N/A
2019/0019564	12/2018	Li et al.	N/A	N/A
2019/0035449	12/2018	Saida et al.	N/A	N/A
2019/0043560	12/2018	Sumbul et al.	N/A	N/A
2019/0065151	12/2018	Chen et al.	N/A	N/A
2019/0073564	12/2018	Saliou	N/A	N/A
2019/0073565	12/2018	Saliou	N/A	N/A
2019/0088329	12/2018	Tiwari et al.	N/A	N/A
2019/0102170	12/2018	Chen et al.	N/A	N/A
2019/0138892	12/2018	Kim et al.	N/A	N/A
2019/0148393	12/2018	Lue	N/A	N/A
2019/0164044	12/2018	Song et al.	N/A	N/A
2019/0164617	12/2018	Tran et al.	N/A	N/A
2019/0213234	12/2018	Bayat et al.	N/A	N/A
2019/0220249	12/2018	Lee et al.	N/A	N/A
2019/0244662	12/2018	Lee et al.	N/A	N/A
2019/0286419	12/2018	Lin et al.	N/A	N/A
2019/0311243	12/2018	Whatmough et al.	N/A	N/A
2019/0311749	12/2018	Song et al.	N/A	N/A
2019/0325959	12/2018	Bhargava et al.	N/A	N/A
2019/0340497	12/2018	Baraniuk et al.	N/A	N/A
2019/0349426	12/2018	Smith et al.	N/A	N/A
2019/0363131	12/2018	Torng et al.	N/A	N/A

2020/0026993	12/2019	Otsuka	N/A	N/A
2020/0065650	12/2019	Tran et al.	N/A	N/A
2020/0075099	12/2019	Choi et al.	N/A	N/A
2020/0098784	12/2019	Nagashima et al.	N/A	N/A
2020/0098787	12/2019	Kaneko	N/A	N/A
2020/0110990	12/2019	Harada et al.	N/A	N/A
2020/0117986	12/2019	Burr et al.	N/A	N/A
2020/0118638	12/2019	Leobandung et al.	N/A	N/A
2020/0143248	12/2019	Liu et al.	N/A	N/A
2020/0160165	12/2019	Sarin	N/A	N/A
2020/0227432	12/2019	Lai et al.	N/A	N/A
2020/0334015	12/2019	Shibata et al.	N/A	N/A
2020/0335146	12/2019	Alrod et al.	N/A	N/A
2020/0343252	12/2019	Lai et al.	N/A	N/A
2020/0365611	12/2019	Hung et al.	N/A	N/A
2020/0381450	12/2019	Lue et al.	N/A	N/A
2020/0402997	12/2019	Ahn et al.	N/A	N/A
2021/0209468	12/2020	Matsumoto et al.	N/A	N/A
2021/0240945	12/2020	Strachan et al.	N/A	N/A
2021/0280250	12/2020	Advani et al.	N/A	N/A
2021/0295922	12/2020	Huang et al.	N/A	N/A
2022/0068386	12/2021	Tseng et al.	N/A	N/A
2022/0093184	12/2021	Kim et al.	N/A	N/A
2022/0122665	12/2021	De Santis et al.	N/A	N/A
2022/0284657	12/2021	Müller et al.	N/A	N/A
2022/0359011	12/2021	Betz et al.	N/A	N/A
2023/0063076	12/2022	Choi	N/A	N/A
2023/0073272	12/2022	Okajima et al.	N/A	N/A
2023/0101654	12/2022	Nava Rodriguez et al.	N/A	N/A
2024/0064044	12/2023	Liu	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
101432821	12/2008	CN	N/A
1998012	12/2009	CN	N/A
103778468	12/2013	CN	N/A
105718994	12/2015	CN	N/A
105789139	12/2015	CN	N/A
106530210	12/2016	CN	N/A
106815515	12/2016	CN	N/A
107077879	12/2016	CN	N/A
107368892	12/2016	CN	N/A
107533459	12/2017	CN	N/A
107767905	12/2017	CN	N/A
108268946	12/2017	CN	N/A
109360593	12/2018	CN	N/A
110598752	12/2018	CN	N/A
113544784	12/2020	CN	N/A
2048709	12/2008	EP	N/A
H0451382	12/1991	JP	N/A
2006127623	12/2005	JP	N/A

2009080892	12/2008	JP	N/A
2013030254	12/2012	JP	N/A
2015523670	12/2014	JP	N/A
2023044545	12/2022	JP	N/A
533528	12/2002	TW	N/A
I310559	12/2008	TW	N/A
201108230	12/2010	TW	N/A
201117213	12/2010	TW	N/A
201523838	12/2014	TW	N/A
201618284	12/2015	TW	N/A
201639206	12/2015	TW	N/A
201715525	12/2016	TW	N/A
201732824	12/2016	TW	N/A
201741943	12/2016	TW	N/A
201802800	12/2017	TW	N/A
201807807	12/2017	TW	N/A
201822203	12/2017	TW	N/A
201939717	12/2018	TW	N/A
202004573	12/2019	TW	N/A
202011285	12/2019	TW	N/A
I696077	12/2019	TW	N/A
202046179	12/2019	TW	N/A
I718566	12/2020	TW	N/A
202122994	12/2020	TW	N/A
I757645	12/2021	TW	N/A
202305804	12/2022	TW	N/A
2012009179	12/2011	WO	N/A
2012015450	12/2011	WO	N/A
2016060617	12/2015	WO	N/A
2016084336	12/2015	WO	N/A
2017091338	12/2016	WO	N/A
2018201060	12/2017	WO	N/A

OTHER PUBLICATIONS

Anonymous, “Data in the Computer”, May 11, 2015, pp. 1-8,

<https://web.archive.org/web/20150511143158/https://homepage.cs.uri.edu/faculty/wolfe/book/Readings/Reading02.htm> (Year: 2015)—See Office Action dated Aug. 17, 2020 in U.S. Appl. No. 16/279,494 for relevance—no year provided by examiner. cited by applicant

Aritome, et al., “Reliability issues of flash memory cells,” Proc. of the IEEE, vol. 81, No. 5, May 1993, pp. 776-788. cited by applicant

Beasley, “Manufacturing Defects,” may be found at <https://slideplayer.com/slide/11407304>, downloaded May 20, 2020, 61 pages. cited by applicant

Chen et al., “A Highly Pitch Scalable 3D Vertical Gate (VG) NAND Flash Decoded by a Novel Self-Aligned Independently Controlled Double Gate (IDG) StringSelect Transistor (SSL),” 2012 Symp. on VLSI Technology (VLSIT), Jun. 12-14, 2012, pp. 91-92. cited by applicant

Chen et al., “Eyeriss: An Energy-Efficient reconfigurable accelerator for deep convolutional neural networks,” IEEE ISSCC, Jan. 31-Feb. 4, 2016, 3 pages. cited by applicant

Choi et al., “Performance Breakthrough in NOR Flash Memory with Dopant-Segregated Schottky-Barrier (DSSB) SONOS Device”, 2009 Symposium on VLSI Technology Digest of Technical Papers, Jun. 16-18, 2009, pp. 1-2. cited by applicant

Entegris FAQ Series, “Contamination Control in Ultrapure Chemicals and Water: Fundamentals of

Contamination Control,” may be found at <https://www.entegris.com/en/home/resources/technical-information/faq/contamination-control-in-ultrapure-chemicals-and-water.html>., downloaded May 20, 2020, 10 pages. cited by applicant

Fukuzumi et al. “Optimal Integration and Characteristics of Vertical Array Devices for Ultra-High Density, Bit-Cost Scalable Flash Memory,” IEEE Dec. 2007, pp. 449-452. cited by applicant

Gonugondla et al., “Energy-Efficient Deep In-memory Architecture for NAND Flash Memories,” IEEE International Symposium on Circuits and Systems (ISCAS), May 27-30, 2018, 5 pages. cited by applicant

Guo et al., “Fast, energy-efficient, robust, and reproducible mixed-signal neuromorphic classifier based on embedded NOR flash memory technology,” IEEE Int’l Electron Devices Mtg., San Francisco, CA, Dec. 2-6, 2017, 4 pages. cited by applicant

Hsu et al., “Study of Sub-30nm Thin Film Transistor (TFT) Charge-Trapping (CT) Devices for 3D NAND Flash Application,” 2009 IEEE, Dec. 7-9, 2009, pp. 27.4.1-27.4.4. cited by applicant

Hubert et al., “A Stacked SONOS Technology, Up to 4 Levels and 6nm Crystalline Nanowires, With Gate-All-Around on Independent Gates (Flash), Suitable for Full 3D Integration,” IEEE 2009, Dec. 7-9, 2009, pp. 27.6.1-27.6.4. cited by applicant

Hung et al., “A highly scalable vertical gate (VG) 3D NAND Flash with robust program disturb immunity using a novel PN diode decoding structure,” 2011 Symp. on VLSI Technology (VLSIT), Jun. 14-16, 2011, pp. 68-69. cited by applicant

IMEC Magazine, Mar. 2018, 35 pages. cited by applicant

Jang et al., “Vertical Cell Array Using TCAT (Terabit Cell Array Transistor) Technology for Ultra High Density NAND Flash Memory,” 2009 Symposium on VLSI Technology Digest of Technical Papers, Jun. 16-18, 2009, pp. 192-193. cited by applicant

Johnson et al., “512-Mb PROM With a Three-Dimensional Array of Diode/Antifuse Memory Cells,” IEEE Journal of Solid-State Circuits, vol. 38, No. 11, Nov. 2003, pp. 1920-1928. cited by applicant

Jung et al, “Three Dimensionally Stacked NAND Flash Memory Technology Using Stacking Single Crystal Si Layers on ILD and TANOS Structure for Beyond 30nm Node,” International Electron Devices Meeting, 2006. IEDM '06, Dec. 11-13, 2006, pp. 1-4. cited by applicant

Katsumata et al., “Pipe-shaped BiCS flash memory with 16 stacked layers and multi-level-cell operation for ultra high density storage devices,” 2009 Symp. on VLSI Technology, Jun. 16-18, 2009, 2 pages. cited by applicant

Kim et al. “Novel Vertical-Stacked-Array-Transistor (VSAT) for Ultra-High-Density and Cost-Effective NAND Flash Memory Devices and SSD (Solid State Drive)”, Jun. 2009 Symposium on VLSI Technology Digest of Technical Papers, pp. 186-187. cited by applicant

Kim et al., “Multi-Layered Vertical Gate NAND Flash Overcoming Stacking Limit for Terabit Density Storage,” 2009 Symposium on VLSI Technology Digest of Technical Papers, Jun. 16-18, 2009, pp. 188-189. cited by applicant

Kim et al., “Novel 3-D Structure for Ultra High Density Flash Memory with VRAT (Vertical-Recess-Array-Transistor) and PIPE (Planarized Integration on the same PlanE),” IEEE 2008 Symposium on VLSI Technology Digest of Technical Papers, Jun. 17-19, 2008, pp. 122-123. cited by applicant

Kim et al., “Three-Dimensional NAND Flash Architecture Design Based on Single-Crystalline STacked ARray,” IEEE Transactions on Electron Devices, vol. 59, No. 1, pp. 35-45, Jan. 2012. cited by applicant

Kim, “Abrasive for Chemical Mechanical Polishing. Abrasive Technology: Characteristics and Applications,” Book Abrasive Technology: Characteristics and Applications, Mar. 2018, 20 pages. cited by applicant

Lai et al. “Highly Reliable MA BE-SONOS (Metal-Al₂O₃ Bandgap Engineered SONOS) Using a SiO₂ Buffer Layer,” VLSI Technology, Systems and Applications 2008, VLSI-TSA International Symposium on Apr. 21-23, 2008, pp. 58-59. cited by applicant

Lai et al., “A Multi-Layer Stackable Thin-Film Transistor (TFT) NAND-Type Flash Memory,” Electron Devices Meeting, 2006, IEDM '06 International, Dec. 11-13, 2006, pp. 1-4. cited by applicant

Liu et al., “Parallelizing SRAM Arrays with Customized Bit-Cell for Binary Neural Networks,” 55th ACM/ESDA/IEEE Design Automation Conference (DAC), Sep. 20, 2018, 4 pages. cited by applicant

Lue et al., “A Highly Scalable 8-Layer 3D Vertical-Gate (VG) TFT Nand Flash Using Junction-Free Buried Channel BE-SONOS Device”, 2010 Symposium on VLSI Technology Digest of Technical Papers, pp. 131-132, Jun. 15-17, 2010. cited by applicant

Lue et al., “A Novel 3D AND-type NVM Architecture Capable of High-density, Low-power In-Memory Sum-of-Product Computation for Artificial Intelligence Application,” IEEE VLSI, Jun. 18-22, 2018, 2 pages. cited by applicant

Lue et al., “A Novel Buried-Channel FinFET BE-SONOS NAND Flash with Improved Memory Window and Cycling Endurance”, 2009 Symposium on VLSI Technology Digest of Technical Papers, p. 224-225. cited by applicant

Meena, et al., “Overview of emerging nonvolatile memory technologies,” Nanoscale Research Letters 9:526, Oct. 2, 2014, 34 pages. cited by applicant

Merrikh-Bayat et al., “High-Performance Mixed-Signal Neurocomputing with Nanoscale Flowing-Gate Memory Cell Arrays,” in IEEE Transactions on Neural Networks and Learning Systems, vol. 29, No. 10, Oct. 2018, pp. 4782-4790. cited by applicant

Minghao Qi, “Ece 695Q Lecture 10: Optical Lithography—Resolution Enhancement Techniques,” may be found at <https://nanohub.org/resources/15325/watch?resid=24507>, Spring 2016, 35 pages. cited by applicant

Nowak et al., “Intrinsic fluctuations in Vertical NAND flash memories,” 2012 Symposium on VLSI Technology, Digest of Technical Papers, pp. 21-22, Jun. 12-14, 2012. cited by applicant

Ohzone et al., “Ion-Implanted Thin Polycrystalline-Silicon High-Value Resistors for High-Density Poly-Load Static RAM Applications,” IEEE Trans. on Electron Devices, vol. ED-32, No. 9, Sep. 1985, 8 pages. cited by applicant

Paul et al., “Impact of a Process Variation on Nanowire and Nanotube Device Performance”, IEEE Transactions on Electron Devices, vol. 54, No. 9, Sep. 2007, p. 2369-2376. cited by applicant

Rincon-Mora, et al., “Bandgaps in the crosshairs: What's the trim target?” IEEE, The Georgia Tech Analog & Power IC Labroator, Oct. 18, 2006, 5 pages. cited by applicant

Rod Nussbaumer, “How is data transmitted through wires in the computer?”, Aug. 27, 2015, pp. 1-3, <https://www.quora.com/How-is-data-transmitted-through-wires-in-the-computer> (Year: 2015)—See Office Action dated Aug. 17, 2020 in U.S. Appl. No. 16/279,494 for relevance—no year provided by examiner. cited by applicant

Sakai et al., “A Buried Giga-Ohm Resistor (BGR) Load Static RAM Cell,” IEEE Symp. on VLSI Technology, Digest of Papers, Sep. 10-12, 1984, 2 pages. cited by applicant

Schuller et al., “Neuromorphic Computing: From Materials to Systems Architecture,” US Dept. of Energy, Oct. 29-30, 2015, Gaithersburg, MD, 40 pages. cited by applicant

Scott Thornton, “What is DRAM (Dynamic Random Access Memory) vs SRAM?”, Jun. 22, 2017, pp. 1-11, <https://www.microcontrollertips.com/dram-vs-sram/> (Year: 2017)—See Office Action dated Aug. 17, 2020 in U.S. Appl. No. 16/279,494 for relevance—no year provided by examiner. cited by applicant

Seo et al., “A Novel 3-D Vertical FG NAND Flash Memory Cell Arrays Using the Separated Sidewall Control Gate (S-SCG) for Highly Reliable MLC Operation,” 2011 3rd IEEE International Memory Workshop (IMW), May 22-25, 2011, 4 pages. cited by applicant

Soudry, et al. “Hebbian learning rules with memristors,” Center for Communication and Information Technologies CCIT Report #840, Sep. 1, 2013, 16 pages. cited by applicant

Tanaka et al., “Bit Cost Scalable Technology with Punch and Plug Process for Ultra High Density Flash Memory,” VLSI Technology, 2007 IEEE Symposium on Jun. 12-14, 2007, pp. 14-15. cited by applicant

The Nikon eReview, “KLA-Tencor Research Scientist Emphasizes Stochastic Challenges at LithoVision 2018,” may be found at <https://nikonereview.com/2018/kla-tencor-research-scientist-emphasizes-stochastic-challenges-at-lithovision-Spring-2018>, 7 pages. cited by applicant

Tseng et al., “In-Memory Approximate Computing Architecture Based on 3D-NAND Flash Memories,” 2022 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits), Honolulu, HI, USA, Jun. 2022, pp. 270-271. cited by applicant

Tseng et al., “In-Memory-Searching Architecture Based on 3D-NAND Technology with Ultra-high

Parallelism,” 2020 IEEE International Electron Devices Meeting (IEDM), San Francisco, CA, USA, Dec. 2020, pp. 36.1.1-36.1.4. cited by applicant

Wang et al., “Three-Dimensional NAND Flash for Vector-Matrix Multiplication,” IEEE Trans. on Very Large Scale Integration Systems (VLSI), vol. 27, No. 4, Apr. 2019, 4 pages. cited by applicant

Wang, Michael, “Technology Trends on 3D-NAND Flash Storage”, Impact 2011, Taipei, dated Oct. 20, 2011, found at www.impact.org.tw/2011/Files/NewsFile/201111110190.pdf. cited by applicant

Webopedia, “DRAM—dynamic random access memory”, Jan. 21, 2017, pp. 1-3, <https://web.archive.org/web/20170121124008/https://www.webopedia.com/TERM/D/DRAM.html> (Year: 2017)—See Office Action dated Aug. 17, 2020 in U.S. Appl. No. 16/279,494 for relevance—no year provided by examiner. cited by applicant

Webopedia, “SoC”, Oct. 5, 2011, pp. 1-2, <https://web.archive.org/web/20111005173630/https://www.webopedia.com/TERM/S/SoC.html> (Year: 2011)—See Office Action dated Aug. 17, 2020 in U.S. Appl. No. 16/279,494 for relevance—No month provided by examiner. cited by applicant

Li et al., “1Mb 0.41 um² 2T-2R cell nonvolatile TCAM with two-bit encoding and clocked self-referenced sensing,” 2013 Symposium on VLSI Technology, Kyoto, Japan, Jun. 11-13, 2013, 12 pages. cited by applicant

Webopedia, “volatile memory”, Oct. 9, 2017, pp. 1-4, https://web.archive.org/web/20171009201852/https://www.webopedia.com/TERMN/volatile_memory.html (Year: 2017)—See Office Action dated Aug. 17, 2020 in U.S. Appl. No. 16/279,494 for relevance—no year provided by examiner. cited by applicant

Whang, SungJin et al. “Novel 3-dimensional Dual Control-gate with Surrounding Floating-gate (DC-SF) NAND flash cell for 1Tb file storage application,” 2010 IEEE Int’l Electron Devices Meeting (IEDM), Dec. 6-8, 2010, 4 pages. cited by applicant

Y.X. Liu et al., “Comparative Study of Tri-Gate and Double-Gate-Type Poly-Si Fin-Channel Spli-Gate Flash Memories,” 2012 IEEE Silicon Nanoelectronics Workshop (SNW), Honolulu, HI, Jun. 10-11, 2012, pp. 1-2. cited by applicant

Primary Examiner: Le; Thong Q

Attorney, Agent or Firm: Haynes Beffel & Wolfeld LLP

Background/Summary

BACKGROUND

Field

(1) This disclosure relates to 3D in-memory searching (IMS), implementable using 3D-NAND memory and as applicable to big data and/or artificial intelligence (AI) processing.

Description of Related Art

(2) Growth of big data and AI (such as AI hardware accelerators) are increasing the importance of searching, comparing, and/or sorting data. Conventional systems implement concurrent searching using ternary content addressable memory (TCAM) technology.

(3) Conventional TCAM technology is implementable using static randomly accessible memory (SRAM) techniques, and thus has relatively low memory density (such as 16 transistors to form a single TCAM cell) and relatively high power usage.

(4) Recently proposed TCAM technology is implementable using nonvolatile memory techniques, such as based on two transistor two resistor (2T2R) techniques and two ferroelectric field-effect transistor (2FeFET) techniques. However, the nonvolatile-implemented TCAM techniques require paired memory for a single TCAM cell (or paired memory cells to implement one search bit or one

data bit) and therefore efficiency of searching and data storage is limited.

(5) Example techniques using NAND-flash-based IMS functions enabled to perform exact matching and approximate matching operations are described in P. H. Tseng et al., In-Memory-Searching Architecture Based on 3D-NAND Technology with Ultra-high Parallelism, 2020 IEDM; and P. H. Tseng et al., In-Memory Approximate Computing Architecture Based on 3D-NAND Flash Memories, 2022 VLSI.

(6) Thus, techniques are needed that enable high performance searching, comparing, and/or sorting, such as relating to big data and/or AI processing.

SUMMARY

(7) A system of one or more computers is configurable to perform particular operations or actions by virtue of having software, firmware, hardware, or a combination of them installed on the system that in operation causes or cause the system to perform the actions. One or more computer programs are configurable to perform particular operations or actions by virtue of including instructions that, when executed by data processing apparatus, cause the apparatus to perform the actions.

(8) A first aspect includes a computing system having a plurality of memories. Each of memory optionally includes a respective match line coupled to a respective sense amplifier, a respective source line, and a respective plurality of memory strings each coupled to (a) the respective match line and (b) the respective source line, and each memory string optionally includes a respective plurality of series-connected floating-gate transistors each having and being responsive to (c) a respective configured state and (d) a respective control input. The engine also includes a plurality of word lines each coupled to a respective one of the respective control inputs of each of the series-connected floating-gate transistors; and a search encoder enabled to receive a search and drive the word lines according to a search encoding; where (e) each memory string is enabled to couple the respective match and source lines that the respective memory string is coupled to via an a match-found impedance responsive to the configured states of the memory string exactly matching the control inputs of the series-connected floating-gate transistors of the memory string, and (f) each sense amplifier is enabled to generate a respective indication of whether the match line it is coupled to is coupled to the source line via the match-found impedance; and where (g) each memory string is further enabled to couple the respective match and source lines that the memory string is coupled to via a no-match-found impedance responsive to the configured states of the memory string not exactly matching the control inputs of the series-connected floating-gate transistors of the memory string, and (h) each sense amplifier is further enabled to generate a respective indication of whether the match line it is coupled to is coupled to the source line via the no-match-found impedance.

(9) Other embodiments of the first aspect include corresponding computer systems, apparatus, and computer programs recorded on one or more computer storage devices, each configured to perform the actions of the methods.

(10) Some computing systems optionally include one or more of the following features. The computing system where the word lines are operated in pairs according to the search encoding and the configured states are managed in pairs according to a feature encoding. Each of the configured states corresponds to a respective threshold voltage. Each of the respective control inputs is coupled to a respective gate of one more of the series-connected floating-gate transistors. The threshold voltages are settable via one or more programming operations. Each of the memory strings optionally includes a respective memory string enable device responsive to a respective memory string enable control input to selectively enable or disable the respective memory string independently of the word lines. Implementations of the described techniques optionally includes hardware, a method or process, or computer software on a computer-accessible medium.

(11) A second aspect includes a method that includes receiving one or more search parameters; encoding the search parameters according to a particular one of a plurality of search encodings to generate pairs of word line voltages, the particular search encoding enabling a search for a match with a feature stored in a pair of floating-gate transistors according to a particular one of a plurality of feature encodings, and where the particular search encoding and the particular feature encoding

enable representation of a one, a zero, and either a one or a zero; driving pairs of word lines of a plurality of series-connected instances of the pair of floating-gate transistors according to the generated word line voltages, the series-connected instances organized as respective pluralities of memory strings operable in parallel to affect a plurality of shared match lines, each shared match line coupled to two or more of the memory strings, and each pair of word lines coupled to at least two of the instances of the pair of floating-gate transistors; sensing voltage changes on the shared match lines to determine which of the shared match lines have a voltage change greater than a predetermined sense threshold; and providing results of the determining as indications of relative matching between the search parameters and the stored features. Other embodiments of the second aspect include corresponding computer systems, apparatus, and computer programs recorded on one or more computer storage devices, each configured to perform the actions of the methods.

(12) Some methods optionally include one or more of the following features. The method where at least one of the plurality of search encodings or at least one of the plurality of feature encodings is in accordance with at least one of an expansion encoding and a one-hot encoding. At least one of the plurality of search encodings or at least one of the plurality of feature encodings is in accordance with a reverse expansion encoding. At least one of the stored features corresponds to a range stored using a one-hot encoding. At least one of the pairs of word line voltages is in accordance with a range encoded as a one-hot encoding. At least one of the plurality of search encodings or at least one of the plurality of feature encodings is in accordance with a low effect encoding. Implementations of the described techniques optionally include hardware, a method or process, or computer software on a computer-accessible medium.

(13) A third aspect includes a computing system that includes interface circuitry to enable a host agent to provide a search and to receive one or more results of the search; a plurality of blocks of memory strings, each block optionally includes a respective plurality of memory strings each receiving a same set of word lines having voltages determinable responsive to the search; a plurality of sense amplifiers each coupled to a corresponding one of the memory strings from each of the blocks; and priority encoder circuitry enabled to receive match indications from the sense amplifiers; where the sense amplifiers are enabled to determine, with respect to matches between (1) feature information stored in the memory strings coupled to the sense amplifier and (2) the word lines received by the memory strings coupled to the sense amplifier, that (3) there are no matches and (4) there is at least one match; and where the priority encoder circuitry, based on the determinations of the sense amplifiers, is enabled to indicate at least a portion of the results as a highest priority match of any matches identified by the sense amplifiers. Some computing systems include corresponding computer systems, apparatus, and computer programs recorded on one or more computer storage devices, each configured to perform the actions of the methods.

(14) Implementations optionally include one or more of the following features. The computing system optionally includes the host agent. The priority encoder circuitry, based on the determinations of the sense amplifiers, is enabled to indicate at least a portion of the results as a second highest priority match of any matches identified by the sense amplifiers. The feature information is stored in the memory strings by configuring threshold voltages of floating-gate transistors of the memory strings. Two or more of the blocks are operable concurrently. A feature encoding is used to encode at least some of the feature information and a search encoding is used to encode at least a portion of the search. The feature encoding is at least one of an expansion encoding, a reverse expansion encoding, a one-hot encoding, and a low effect encoding. The computing system is part of a system-on-a-chip (SOC). Implementations of the described techniques optionally include hardware, a method or process, or computer software on a computer-accessible medium.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

- (1) FIG. 1A, FIG. 1B, and FIG. 1C collectively illustrate various aspects of an example 3D (search) engine.
- (2) FIG. 1D illustrates an example 2D (search) engine and use thereof.
- (3) FIG. 1E illustrates an expansion encoding usage example of a 2D (search) engine of a 3D (search) engine.
- (4) FIG. 1F illustrates a one-hot encoding usage example of a 2D (search) engine of a 3D (search) engine.
- (5) FIG. 2 illustrates an example expansion encoding for a 3D (search) engine.
- (6) FIG. 3 illustrates an example one-hot encoding for a 3D (search) engine.
- (7) FIG. 4A illustrates an example expansion encoding and a corresponding example low effect encoding.
- (8) FIG. 4B illustrates an example expansion encoding with Hamming distances compared to an input of 1.
- (9) FIG. 4C illustrates example threshold voltages and word line voltages that enable comparing of ones, zeros, and don't cares.
- (10) FIG. 4D illustrates icons relating high and low gate overdrive to high and low cell current capability.
- (11) FIG. 4E illustrates floating-gate transistor impedance as a function of search information versus stored data.
- (12) FIG. 5A illustrates an example reverse expansion encoding for search inputs of a 3D (search) engine.
- (13) FIG. 5B graphically illustrates, for reverse expansion encoding, example Hamming distances as a function of stored data values and search input values.
- (14) FIG. 6 graphically illustrates, for expansion encoding, example Hamming distances as a function of stored data values and search input values.
- (15) FIG. 7A illustrates an example of one-hot encoding with stored ranges.
- (16) FIG. 7B illustrates a one-hot encoding on input range example.
- (17) FIG. 8 illustrates an example operating flow for a 3D (search) engine.
- (18) FIG. 9 illustrates an example 3D engine appliance comprising an example 3D engine device.
- (19) FIG. 10 illustrates an example hardware system having a 3D (search) Engine.

DETAILED DESCRIPTION

- (20) A detailed description of techniques relating to architecture for a 3D search engine is provided with reference to FIGS. 1A-10.
- (21) One or more flow diagrams are described herein. Processing described by the flow diagrams is implementable and/or directable using processors programmed using computer programs stored in memory accessible to computer systems and executable by the processors, using dedicated logic hardware (including field programmable integrated circuits), and using various combinations thereof. Various actions are combinable, performable in parallel, and/or performable in a different sequence without affecting processing achieved. In some cases, a rearrangement of actions achieves identical results only if certain other changes are made as well. In other cases, a rearrangement of actions achieves identical results only if certain conditions are satisfied. Furthermore, for clarity, some of the flow diagrams herein omit certain some actions not necessary for understanding the disclosed techniques. Various additional actions are performable before, after, and/or between the illustrated actions.
- (22) Examples of selected acronyms, mnemonics, and abbreviations used in the description are as follows.
- (23) TABLE-US-00001 Acronym/Mnemonic/ Abbreviation Example 2FeFET Two Ferroelectric Field Effect Transistor 2T0C (DRAM- Two-Transistor Zero-Capacitor like device) 2T2R Two Transistor Two Resistor AI Artificial Intelligence BL Bit Line CSL Common Source Line DNA

Deoxyribonucleic Acid Random Access read/write Memory FeFET Ferroelectric Field-Effect Transistor GSL Ground Select Line HD Hamming Distance IMS In-Memory Searching NAND Not AND; e.g., series connection of devices forming a memory string SA Sense Amplifier SLC Single-Level Cell SONOS (memory) Silicon-Oxide-Nitride-Oxide-Silicon (memory) SRAM Static Random Access read/write Memory SSD Solid State Drive SSL String Select Line TCAM Ternary Content Accessible Memory WL Word Line

(24) An example of a memory string is a series-connected plurality of memory devices. An example of a memory device is an element enabled to store information to indicate one of at least two mutually exclusive states of the memory device. The states are settable via programming the memory device and are readable via activating a control input of the memory device. In some types of memory devices, the programming is via configuring a threshold voltage of the memory device. The configuring is sometimes referred to as programming the memory device (e.g., to a one or a zero). Sometimes the configuring is simply referred to as storing the value (e.g., a one or a zero).

3D (Search) Engine Concepts

(25) Disclosed herein is a novel 3D search engine architecture implementable using 3D-NAND memory that enables high performance searching, comparing, and/or sorting, such as relating to big data and/or AI processing. The architecture provides for low latency, high resolution, high content density, multi-block functionality, and robust reliability. The architecture is applicable to various fields such as big-data searching, AI hardware accelerators and/or classifiers, approximate computing, associative memory, few-shot learning, SSD data management, DNA matching, data filtering, hyper-dimensional computing, as well as other applications benefiting from IMS enabled for long search words and large data capacities. Since the architecture enables high performance comparing and sorting, as well as searching, the term “search” is parenthetical as in “3D (search) engine” as well as in “2D (search) engine”.

(26) A novel 3D (search) engine receives searches for application to word lines of a nonvolatile memory array. The 3D (search) engine uses two word lines per bit of information of the searches. The 3D (search) engine uses respective bit lines of the nonvolatile memory array as respective matching lines for searching. Respective memory strings (e.g., NAND memory strings) of the nonvolatile memory array are usable to store respective data words, e.g., corresponding to features to search for. Respective pluralities of the memory strings are coupled in parallel to respective shared bit lines.

(27) In some usage scenarios, read latency of a 3D (search) engine is reduced (at the potential expense of a relatively large current transient) by operating multiple blocks concurrently. The concurrent operating is via switching a plurality of activation controls, e.g., string select lines (SSLs).

(28) In some usage scenarios, a 3D (search) engine is used with expansion encoding and/or one-hot encoding to facilitate, e.g., optional range searching capability.

(29) A 3D (search) engine has specific applicability to big data and/or AI, such as for various searching, comparing, and/or sorting operations. A host agent directs storage of features in the 3D (search) engine and then provides searches to the 3D (search) engine to determine matches to the stored features. Indications of the matches are returned to the host agent.

(30) A 3D (search) engine uses a plurality of 2D (search) engines to perform operations concurrently across the 2D (search) engines and can be referred to as a “Search Cube” or simply a “Cube”. The 3D (search) engine is enabled to perform a plurality of searches concurrently using a plurality of search parameters across a plurality of stored features. In some usage contexts, the concurrent searching is per 2D (search) engine. For instance, each 2D (search) engine is tasked with a unique search operation. In some usage contexts, the concurrent searching is within a particular 2D (search) engine. For instance, different portions of the particular 2D (search) engine are tasked with a unique search operation. In some usage contexts, the concurrent searching is parallelized across a plurality of 2D (search) engines and within one or more of the 2D (search) engines. Indications of matches between the search parameters and the stored features are provided by the 2D (search) engines for processing. The processing includes, for example, determining an overall similarity between the search parameters and the stored features and/or identifying which one or more stored features that are

similar compared to the search parameters, e.g., via one or more of buffering, caching, and/or priority encoding (circuitry).

(31) Each 2D (search) engine used by the 3D (search) engine comprises a plurality of memory devices, to detect if there is a match between search information provided to the 2D (search) engines as determined from the search parameters and features stored in the 2D (search) engines. The 2D (search) engines generate match indications responsive to the detecting and provide the match indications for buffering, caching, priority encoding, and output for usage in a system. The features are stored according to a feature storage encoding. The search information is encoded according to a search encoding. Examples of the storage and feature encodings include expansion encoding, one-hot encoding, low effect encoding, and reverse expansion encoding. The 3D (search) engine is enabled to selectively perform exact and approximate matching, such as according to storage and/or feature encoding. An exact match corresponds to all bits matching, such as all bits of search parameters matching all bits of a value stored in a memory string. An approximate match corresponds to less than all bits matching, such as all but one bit matching, all but two bits matching, or alternatively a fraction of the bits matching, such as 90% of the bits matching (e.g., 9 out of 10 bits match).

(32) A control agent of the 3D (search) engine selectively operates the 2D (search) engines concurrently. The concurrent operation enables concurrent match determinations against a same set of search information provided to the 2D (search) engines operating concurrently. Each 2D (search) engine generates a match indication according to the search information and the features stored in the 2D (search) engine. A priority encoder of the 3D (search) engine processes the match indications to determine relative matching between the 2D (search) engines and/or within one or more of the 2D (search) engines.

(33) A 2D (search) engine has a number of memory strings operable concurrently. The memory strings of a 2D (search) engine are coupled in parallel between a match line and a source line of the 2D (search) engine. Each of the memory strings is usable to store a respective feature, each feature having a plurality of bits and stored according to a selected one of the feature encodings. Each of the memory strings is usable to compare an (encoded) search input with an (encoded) feature stored in the memory string. Prior to searching, the control agent performs pair-wise programming of the features into the memory devices according to the selected feature encoding. The pair-wise programming is according to using a series-connected pair of the memory devices for each bit of a feature. For searching, the control agent drives the control inputs in pairs, according to the search information and according to the selected search encoding. The search information comprises a search pattern and optional mask and/or range information, for use in encodings that enable searching according to ranges. Results of the searching are provided as a plurality of match indications that are then optionally buffered, cached, and/or priority encoded.

3D (Search) Engine Example Implementation

(34) An example 3D (search) engine implementation uses floating-gate transistors as memory devices. The floating-gate transistors are instantiated in NAND series-coupled memory strings. Thus, the 3D (search) engine is sometimes referred to as a 3D NAND search engine or more broadly as a search system or alternatively as a computing system. The memory strings are in turn instantiated in parallel between a shared bit line and a shared source line. The memory strings coupled to a shared bit line are sometimes referred to as a 2D (search) engine or more broadly as a memory.

(35) The floating-gate transistors are programmed to store bits of features according to at least two mutually exclusive states. The programming configures the threshold voltages of the floating-gate transistors. The programmed floating-gate transistors are conditionally responsive (e.g., via an impedance change) to indicate a match between a stored feature and a search provided on word lines coupled to control gates of the floating-gate transistors.

(36) The word lines are usable to provide one or more search inputs to search for among the stored features that have been programmed into the floating-gate devices of the memory strings. Each memory string is usable to compare a search value provided via the word lines coupled to the memory string to the stored feature that has been programmed into the memory string.

(37) Each shared bit line is usable by a match line to indicate zero, one, or more matches between one or more search inputs respectively provided to each memory string coupled to the shared bit line and the stored features of the respective memory strings.

(38) Match indications provided by the shared bit lines are priority encoded (e.g., according to a predetermined priority scheme) and one or more matching results are indicated to one or more other agents of a system. Thus, a plurality of 2D (search) results are combined into one or more 3D (search) results.

(39) FIG. 1A-FIG. 1C collectively illustrate various aspects of an example 3D (search) engine, such as a NAND-based 3D (search) engine, as 3D Engine **9000**.

(40) FIG. 1A illustrates an overview of an example 3D (search) engine, FIG. 1B illustrates some aspects of the example 3D (search) engine of FIG. 1A in different detail, and FIG. 1C illustrates some aspects of the example 3D (search) engine of FIG. 1A in more detail.

(41) Returning to FIG. 1A, for clarity, the figure is simplified.

(42) 3D Engine **9000** comprises a plurality (e.g., 128K) of identical (or alternatively substantially similar) 2D (search) engines, as 2D Engines **9050**. For clarity, any details of all but one of the 2D (search) engines are omitted.

(43) Each of the 2D (search) engines of 2D Engines **9050** has a plurality of memory strings, conceptually represented by Memory Strings **9030**, to store feature information provided to 3D Engine **9000**. The stored features are conceptually represented by Stored Features **9900**. Memory Strings **9030** are also used to determine matches between the stored feature information and search information provided to 3D Engine **9000** by conditionally affecting respective match lines of Match Lines **9040**. For example, the conditionally affecting includes discharging a precharged value of a match line based on a match between a portion of the search inputs and a portion of the stored features.

(44) Feature Encoder **9003** receives Features **9001** (e.g., sourced by a host agent) provided to 3D Engine **9000**. In response, Feature Encoder **9003** generates, via a selected feature encoding, Feature Inputs **9004** suitable for application to the memory strings of the 2D (search) engines to store as portions of Stored Features **9900**. Each memory string of each 2D (search) engine is enabled to store unique features, and thus Feature Inputs **9004** is represented as having unique couplings to each of the 2D (search) engines of 2D Engines **9050**. The unique couplings are implementable via separate signals to each of the 2D (search) engines, a shared time-multiplexed set of shared signals (e.g., a bus), or any other suitable communication mechanism that enables each of the memory strings of each of the 2D (search) engines to store unique features.

(45) Search Encoder **9005** receives Searches **9002** (e.g., sourced by the host agent) provided to 3D Engine **9000**. In response, Search Encoder **9005** generates, via a selected search encoding, Search Inputs **9006**, suitable for application to the memory strings of the 2D (search) engines of 2D Engines **9050** to determine matches (if any) between Search Inputs **9006** and Stored Features **9900**. Each 2D (search) engine receives identical search information, and thus Search Inputs **9006** is represented as having parallel couplings to each of the 2D (search) engines. This is in contrast with the unique couplings used for distribution of feature information. The parallel couplings are implementable via separately buffered signals to each of the 2D (search) engines, a single set of coupled signals (e.g., a bus), or any other suitable communication mechanism that enables each of the 2D (search) engines to receive identical search information. Each memory device of each memory string of a particular 2D (search) engine receives a unique word line (as provided via Search Inputs **9006**) as conceptually illustrated in FIG. 1A as a sampling of word lines illustrated as WLS **9010**. While each 2D (search) engine receives identical search information, each of the memory strings within a 2D (search) engine receives unique search information.

(46) Responsive to Search Inputs **9006** and based on Stored Features **9900** of each of the 2D (search) engines, each 2D (search) engine bit line (e.g., each of Match Lines **9040**) provides respective match information to a respective sense amplifier of SAs **9007** to determine match indications for each of the 2D (search) engines. The match indications are then processed by Page Buffer, Cache, Priority

Encoder **9008** and provided as Output Matching Result(s) **9009**, and communicated, e.g., to the host agent as Result(s) **9099**.

(47) Turning now to FIG. **1B**, some aspects the example 3D (search) engine of FIG. **1A** are illustrated in more detail. Elements of FIG. **1B** and FIG. **1A** having identical element identifiers are identical elements. As with FIG. **1A**, for clarity, FIG. **1B** is simplified.

(48) Three of the 2D (search) engines (2D (search) Engine_0 **150**, 2D (search) Engine_128K-2 **158**, and 2D (search) Engine_128K-1 **159**) are illustrated as having respectively concurrently operable memory strings. As summarized with respect to FIG. **1A**, 3D Engine **9000** comprises sense amplifier circuitry. There is one sense amplifier for each 2D (search) engine, collectively SAs **9007**. For clarity, three of the sense amplifiers are illustrated, each coupled to a respective one of the illustrated 2D (search) engines. 3D Engine **9000** further comprises post-sensing circuitry, collectively Page Buffer, Cache, Priority Encoder **9008** to process output(s) of each of the sense amplifiers of SAs **9007** to produce results of matching searches as Output Matching Result(s) **9009**.

(49) Each of the 2D (search) engines are illustrated in the X and Z dimensions. In some variations built as an integrated circuit based on a planar process, layers of the integrated circuit are built up in the Z dimension. The 2D (search) engines are arrayed in the Y dimension.

(50) Turning now to FIG. **1C**, some aspects the example 3D (search) engine of FIG. **1A** and FIG. **1B** are illustrated in more detail. Elements of FIG. **1C**, FIG. **1B**, and FIG. **1A** having identical element identifiers are identical elements. As with FIG. **1A** and FIG. **1B**, for clarity, FIG. **1C** is simplified.

(51) Each of the 2D (search) engines comprises a respective bit line. In particular, 2D (search) Engine_0 **150**, 2D (search) Engine_128K-2 **158**, and 2D (search) Engine_128K-1 **159** respectively comprise BL_0 **140**, BL_128K-2 **148**, and BL_128K-1 **149**. Note that in FIG. **1A**, the bit lines are collectively referred to as Match Lines **9040**, as each bit line is usable to indicate a match between search and stored information. Each 2D (search) engine comprises a plurality of memory strings (e.g., **512** memory strings). For clarity, four of the memory strings are illustrated. Each memory string is selectively coupled to the bit line of the 2D (search) engine via a transistor controlled by a respective SSL input. The transistor and the SSL input collectively enable selectively enabling (disabling) the memory string based on activation (deactivation) of the SSL input. Each SSL input is coupled to a memory string of each of the 2D (search) engines. Thus, each SSL input enables selectively enabling (disabling) a block of the memory strings based on activation (deactivation) of the SSL input.

(52) For instance, SSL0 **170** is enabled to selectively couple a 1st memory string of 2D (search) Engine_0 **150** to BL_0 **140**, a 1st memory string of 2D (search) Engine_128K-2 **158** to BL_128K-2 **148**, and a 1st memory string of 2D (search) Engine_128K-1 **159** to BL_128K-1 **149**, each via a respective transistor. SSL0 **170** is also enabled to selectively couple 128K-3 1st memory strings (not illustrated) of the 2D (search) engines between 2D (search) Engine_0 **150** and 2D (search) Engine_128K-2 **158** respectively to the bit lines therein via respective transistors.

(53) For another instance, SSL511 **179** is enabled to selectively couple a 512th memory string of 2D (search) Engine_0 **150** to BL_0 **140**, a 512th memory string of 2D (search) Engine_128K-2 **158** to BL_128K-2 **148**, and a 512th memory string of 2D (search) Engine_128K-1 **159** to BL_128K-1 **149**, each via a respective transistor. SSL511 **179** is also enabled to selectively couple 128K-3 512th memory strings (not illustrated) of the 2D (search) engines between 2D (search) Engine_0 **150** and 2D (search) Engine_128K-2 **158** respectively to the bit lines therein via respective transistors.

(54) Similar to the bit lines, each of the 2D (search) engines comprises a respective source line (illustrated but for clarity not identified). Each memory string is selectively coupled to the source line of the 2D (search) engine via a transistor controlled by a respective GSL input.

(55) Again similar to the bit lines, for instance, GSL0 **160** is enabled to selectively couple the 1st memory string of 2D (search) Engine_0 **150** to a source line of 2D (search) Engine_0 **150**, the 1st memory string of 2D (search) Engine_128K-2 **158** to a source line of 2D (search) Engine_128K-2 **158**, and the 1st memory string of 2D (search) Engine_128K-1 **159** to a source line of 2D (search) Engine_128K-1 **159**, each via a respective transistor. GSL0 **160** is also enabled to selectively couple 128K-3 1st memory strings (not illustrated) of the 2D (search) engines between 2D (search)

Engine_0 **150** and 2D (search) Engine_128K-2 **158** respectively to source lines therein via respective transistors. The source lines are all connected to ground.

(56) Again similar to the bit lines, for another instance, GSL511 **169** is enabled to selectively couple a 512th memory string of 2D (search) Engine_0 **150** to the source line of 2D (search) Engine_0 **150**, a 512th memory string of 2D (search) Engine_128K-2 **158** to the source line of 2D (search) Engine_128K-2 **158**, and a 512th memory string of 2D (search) Engine_128K-1 **159** to the source line of 2D (search) Engine_128K-1 **159**, each via a respective transistor. GSL511 **169** is also enabled to selectively couple 128K-3 512th memory strings (not illustrated) of the 2D (search) engines between 2D (search) Engine_0 **150** and 2D (search) Engine_128K-2 **158** respectively to the source lines therein via respective transistors.

(57) The SSL and GSL inputs are coupled in parallel to corresponding transistors in each of a corresponding memory string in each of the 2D (search) engines. As there are 512 memory strings in each of the 2D (search) engines, there are 512 SSL inputs and 512 GSL inputs. Note that in FIG. **1C**, the coupling between GSL0 **160** and a transistor of 2D (search) Engine_128K-1 **159** is omitted for clarity, although a dotted-line extension of GSL0 **160** is illustrated.

(58) In 3D Engine **9000**, each of the memory strings is a same length of 128 floating-gate transistors (excluding the transistors that couple each memory string to a bit line and a source line). In a planar integrated circuit process implementation of 3D Engine **9000**, a certain number of layers are used to implement the memory strings. As the length of the memory strings increases, so does the number of layers used to implement the memory strings. Thus, doubling the length of the memory strings doubles the layers used to implement the memory strings.

(59) Turning now to word lines, conceptually each 2D (search) engine is coupled to a same 2D array of word lines. In the X dimension, there are 512 columns of word lines, corresponding one-to-one with each of the 512 memory strings. In the Z dimension, there are 128 rows of word lines, corresponding one-to-one with the floating-gate transistors of each of the memory strings. Thus, the 2D array of word lines is 512 columns by 128 rows or 512*128 individual word lines. Each word line couples to a single floating-gate transistor of a single memory string in each of the 2D (search) engines. Note that in FIG. **1C**, the coupling between WL0_0 **100** and a floating-gate transistor of 2D (search) Engine_128K-1 **159** is omitted for clarity, although a dotted-line extension of WL0_0 **100** is illustrated.

(60) For instance, WL0_0 **100** couples to a floating-gate transistor of the 1st memory string of 2D (search) Engine_0 **150**, a floating-gate transistor of the 1st memory string of 2D (search) Engine_128K-2 **158**, and a floating-gate transistor of the 1st memory string of 2D (search) Engine_128K-1 **159**. WL0_0 **100** also couples to a floating-gate transistor in each of the 128K-3 1st memory strings (not illustrated) of the 2D (search) engines between 2D (search) Engine_0 **150** and 2D (search) Engine_128K-2 **158**.

(61) For another instance, WL0_511 **109** couples to a floating-gate transistor of the 512th memory string of 2D (search) Engine_0 **150**, a floating-gate transistor of the 512th memory string of 2D (search) Engine_128K-2 **158**, and a floating-gate transistor of the 512th memory string of 2D (search) Engine_128K-1 **159**. WL0_511 **109** also couples to a floating-gate transistor in each of the **128K-3** 512th memory strings (not illustrated) of the 2D (search) engines between 2D (search) Engine_0 **150** and 2D (search) Engine_128K-2 **158**.

(62) In this description, word lines are sometimes referred to in a 2D context by a block number and a layer number. Conceptually a block exists in the Y and Z dimensions and a layer exists in the X and Y dimensions. Conceptually the block number corresponds to the column number and the layer number corresponds to a row number.

(63) A block corresponds to memory strings from each of the 2D (search) engines. As a specific example, the memory strings coupled between GSL511 **169** and SSL511 **179** correspond to block number **511**. As another specific example, the memory strings coupled between GSL0 **160** and SSL0 **170** correspond to block number 0.

(64) A layer corresponds to floating-gate transistors from each of the memory strings of all the 2D

(search) engines. As a specific example, the floating-gate transistors coupled to WL0_511 **109** and WL0_0 **100** are part of layer number 0. As another specific example, the floating-gate transistors coupled to WL1_511 **119** are part of layer number 1. Similarly, the floating-gate transistors respectively coupled to WL2_511 **129**, WL126_0 **180**, and WL127_0 **190** are respectively part of layer numbers 2, 3, 126, and 127. Layers also correspond to layers with respect to planar integrated circuit process, although a layer of floating-gate transistors corresponds to one or more planar integrated circuit process layers.

(65) Thus, word lines are identifiable in the Z and X dimensions respectively as “WL” “<layer number>” and “<block number>”. Thus, WL0_0 **100** is of layer number 0 and block number 0. WL0_511 **109** is of layer number 0 and block number **511**.

(66) FIG. **1D** illustrates 2D (search) Engine_0 **150** in the context of portions of FIG. **1A** and FIG. **1B**. Numerous word lines are illustrated and identified in the aforementioned “WL” “<layer number>” and “<block number>” scheme, as illustrated by the icon near WL6_511. 2D (search) Engine_0 **150** comprises 512 memory strings. For clarity, four of the memory strings are illustrated (MemStr0 **130**, MemStr1 **131**, MemStr510 **138**, and MemStr511 **139**). The floating-gate transistors of each of the memory strings are arranged and operated in pairs. An example of one of the pairs is IMS Data Bit **134**. Another example is IMS Data Bit **135**. Conceptually, each of the even-layer-numbered word lines is coupled to one of the floating-gate transistors of the pairs. E.g., WL0_0, WL0_1 . . . WL0_510, and WL0_511 of layer 0 are each coupled to a respective one of the pairs, as are WL2_0 . . . WL2_511 of layer 2, and so forth, to WL126_0 . . . WL126_511 of layer **126**. Further, each of the odd-layer-number word lines are coupled to the other one of the floating-gate transistors of the pairs. E.g., WL1_0, WL1_1 . . . WL1_510, and WL1_511 of layer 1 are coupled respectively to the other floating-gate transistors of the pairs that WL0_0, WL0_1 . . . WL0_510, and WL0_511 of layer 0 are respectively coupled to.

(67) FIG. **1D** also illustrates a specific usage example of a 2D (search) engine.

(68) Each memory string is usable to store a feature. Thus, the **512** memory strings of 2D (search) Engine_0 **150** are collectively usable to store up to 512 features. For clarity, one of the features is illustrated as Feature 0 **1133**. Each of the pairs of floating-gate transistors is usable to store a single bit of one of the features. There are 64 pairs of floating-gate transistors in each of the memory strings. Thus, each of the memory strings is usable to store a feature of up to 64 bits.

(69) Each memory string is usable to compare the feature stored therein with a search input supplied via the word lines coupled to the memory string. Each floating-gate transistor of the memory string is responsive to word line voltage according to the feature bit stored by the floating-gate transistor (e.g., the threshold voltage of the floating-gate transistor configured by programming). For example, operated in pairs, a pair of the floating-gate transistors indicates a match to a search input provided to the pair of floating-gate transistors via a pair of word lines via a match-found impedance (e.g., a relatively low impedance). The pair indicates lack of a match via a no-match-found impedance (e.g., a relatively high impedance). Thus, across an entirety of a memory string, responsive to all pairs of floating-gate transistor stored feature bits matching a multi-bit search input as provided via corresponding pairs of word lines, the entirety of the memory string indicates a corresponding memory string match-found impedance. Responsive to less than all pairs matching, the entirety of the memory string indicates a corresponding memory string impedance that is greater than the memory string match-found impedance. Responsive to no pairs matching, the entirety of the memory string indicates a corresponding memory string no-match-found impedance. In between all bits matching and no bits matching extremes, the memory string indicates a corresponding memory string impedance that monotonically decreases (increases) as correspondingly more (less) stored feature bits that match bit-for-bit compared to applied word line inputs. String Current **136D** indicates current flow in the memory string as based on the indicated memory string impedance.

(70) For example, MemStr0 **130** is coupled to BL_0 **140** via activation of SSL0 and coupled to CSL **145** via activation of GSL0. Feature 0 **1133** stored in MemStr0 **130** is programmed to all zero bits except for a single one bit in the floating-gate transistor pair of IMS Data Bit **135**.

(71) Consider a first operating scenario with respect to the example, with all word line pairs coupled to MemStr0 **130** (WL0_0, WL1_0 . . . WL126_0, and WL127_0) indicating in pairs a search for all zero bits. In response, all floating-gate transistor pairs of MemStr0 **130**, except for the pair of IMS Data Bit 135, indicate the match-found impedance. The pair of IMS Data Bit 135 indicates the no-match-found impedance (since there is no match between the word line search input and the stored feature bits). Thus, MemStr0 **130** overall indicates an impedance that is greater than the memory string match-found impedance. SA **143** senses that the impedance is greater than the memory string match-found impedance and indicates, via Sense Output **144**, that there is not a match between the word line pairs and Feature 0 **1133**.

(72) Consider a second operating scenario with respect to the example, with the word line pair coupled to IMS Data Bit **135** (WL2_0 and WL3_0) indicating a search for a one bit and the remainder of the word line pairs coupled to MemStr0 **130** (WL0_0, WL1_0, WL4_0, WL5_0. . . . WL126_0, and WL127_0) indicating in pairs a search for zero bits. In response, all floating-gate transistor pairs of MemStr0 **130**, including the pair of IMS Data Bit **135**, indicate the match-found impedance. Thus, MemStr0 **130** overall indicates an impedance that is equal to the memory string match-found impedance. SA **143** senses that the impedance is the memory string match-found impedance and indicates, via Sense Output **144**, that there is a match between the word line pairs and Feature 0 **1133**.

(73) SA **143** performs the sensing and generates Sense Output **144**, e.g., by detecting a voltage change greater than (or lesser than) a predetermined sense threshold on BL_0 **140** within a sensing time interval. In some variations, the voltage change is due to MemStr0 **130** discharging a precharged value on BL_0 **140** in accordance with the indicated memory string impedance. Responsive to the first operating scenario, where the impedance is greater than the memory string match-found impedance, BL_0 **140** is discharged by MemStr0 **130** more slowly than compared to the second operating scenario, where the impedance is equal to the memory string match-found impedance. The sense threshold and/or the sensing time interval are set to enable distinguishing between the match-found and other than the match-found impedance based on sense speed (e.g., engine bandwidth and/or latency) and noise immunity (e.g., engine resilience).

(74) For either the first or the second operating scenarios, and continuing in the context of FIG. **1A**, FIG. **1B**, and FIG. **1C**, Sense Output **144** is provided to Page Buffer, Cache, Priority Encoder **9008** along with corresponding sense amplifier information of the other 2D engines. Page Buffer, Cache, Priority Encoder **9008** buffers, caches, and/or priority encodes Sense Output **144** in the context of the entirety of the 2D engine sense outputs.

(75) In some applications, a controller activates multiple blocks of 2D (search) engines to enable searching the multiple blocks concurrently. For example, a plurality of SSL inputs is activated in a same search operation to search multiple blocks during the same search operation.

(76) For instance, the controller activates SSL_511 and SSL_510 in a same search operation to enable searching two blocks concurrently in each of the **128K** 2D (search) engines, 2D (search) Engine_0 **150** . . . 2D (search) Engine_128K-2 **158**, and 2D (search) Engine_128K-1 **159**. Thus, the controller enables 3D Engine **9000** to search 2*128K blocks concurrently. Alternatively, the controller activates any two SSL inputs in a same search operation to enable searching any two blocks concurrently. For other instances, the controller activates any four, six, 8, 32, 64, 128, 256, or 512 SSL inputs in a same search operation to enable searching corresponding multiple blocks concurrently. For example, the controller activates all 512 SSL inputs in a same search operation. Thus, the controller enables 3D Engine **9000** to search 512 blocks concurrently, comprising 512*128K memory strings. Optionally, the controller is responsive to a control register that specifies if and to what extent multiple blocks are searched concurrently.

(77) FIG. **1A**, FIG. **1B**, and FIG. **1C** illustrate an example 3D (search) engine as having a particular number of 2D (search) engines, each of which have a particular number of memory strings, each of which have a particular number of IMS data bits (each as a respective pair of floating-gate transistors). Specifically, 3D Engine **9000** has 128K 2D (search) engines, (2D (search) Engine_0 **150**

... 2D (search) Engine_128K-1 **159**), but some variations have less or more 2D (search) engines. Each 2D (search) engine has **512** memory strings (e.g., MemStr0 **130**. . . . MemStr511 **139**), but some variations have less or more memory strings. Each memory string has 64 IMS data bits (e.g., coupled to word line pairs WL0_1 and WL1_0 . . . WL126_0 and WL 127_0), but some variations have less or more IMS data bits per memory string.

(78) Some variations of 2D (search) engine and associated sense amplifier circuitry enable sense amplifier resolution of a single bit (e.g., match/no-match). Some variations enable sense amplifier resolution of a plurality of bits (e.g., to encode no matches, one match, or two or more matches). Some variations enable sense amplifier indications that are analog rather than digital. For example, a sense amplifier output is a monotonically increasing function based on memory string current, and the memory string current is a monotonically increasing (or decreasing) indicator of similarity between search inputs and stored features.

(79) Some variations of 3D (search) engines enable identifying at most one feature (stored in a single memory string) as matching for a search. Some variations enable concurrently identifying at most a plurality of features (stored in a plurality of memory strings) as matching for a search. Some variations enable concurrently identifying at most one 2D (search) engine as having one or more features stored therein as matching for a search. Some variations enable concurrently identifying a plurality of 2D (search) engines each as having one or more features stored therein as matching for a search.

(80) Some variations of 3D (search) engines enable priority encoding match indications according to predetermined 2D (search) engine priority (e.g., a predetermined 2D (search) engine is a lowest priority and a another predetermined 2D (search) engine is a highest priority). Thus, a match indication from the predetermined highest priority 2D (search) engine corresponds to a highest priority match. Further, a match indication from a predetermined second highest priority 2D (search) engine corresponds to a second highest priority match. Some variations enable priority encoding according to how many stored feature bits within a memory string match respective search input bits (e.g., zero match, one matches, two match, or three or more match). Thus, a highest priority match corresponds to a highest number of stored feature bits within a memory string matching respective search input bits, and a second highest priority match corresponds to a second highest number of stored feature bits within a memory string matching respective search input bits. Some variations enable priority encoding according to how many memory strings match within a particular 2D (search) engine (e.g., zero match, one matches, two match, or three or more match). Thus, a highest priority match corresponds to a highest number of memory strings matching within a particular 2D (search) engine, and a second highest priority march corresponds to a second highest number of memory strings matching within a particular 2D (search) engine.

(81) Some variations enable priority encoding that identifies a single matching stored feature. Some variations enable priority encoding that identifies a plurality of matching stored features. Some variations enable priority encoding that identifies a single “best matching” stored feature, having, e.g., a highest number of stored feature bits matching a search, and the highest number is less than a maximum stored feature length. Some variations enable priority encoding that identifies a plurality of “best matching” stored features, ranked, e.g., according to number of stored feature bits matching a search for each of a plurality of stored features.

(82) In various usage scenarios (e.g., due to features programmed into the 2D (search) engines, provided search information, and/or encodings thereof), various numbers of 2D (search) engines of a 3D (search) engine detect a match between one or more features stored in one or more of the memory strings of the 2D (search) engines and the search information provided to the 2D (search) engines of the 3D (search) engine.

(83) In some usage scenarios, zero, one, or more memory strings of a 2D (search) engine detect matches, and sense amplifier circuitry encodes matching results as a plurality of bits. For example, the sense amplifier circuitry encodes zero memory strings matching, one memory string matching, two memory strings matching, and more than two memory strings matching as four mutually

exclusive encodings. Some variations enable priority encoding according to one or more factors, such as 2D (search) engine priority, number of matching stored feature bits, number of memory strings (e.g., features).

(84) For example, responsive to no matches from any of 2D (search) Engine_0 **150** . . . 2D (search) Engine_128K-1 **159**, SAs **9007** provides a vector of all zeros to Page Buffer, Cache, Priority Encoder **9008**. In response, Page Buffer, Cache, Priority Encoder **9008** priority encodes the all zero vector to a binary value indicating zero matches and provides the binary value via Output Matching Result(s) **9009**.

(85) Continuing with the example, responsive to a single memory string of 2D (search) Engine_0 **150** detecting a match, SAs **9007** provides a vector with an LSB corresponding to 2D (search) Engine_0 **150** as a one and all other bits as zeros. In response, Page Buffer, Cache, Priority Encoder **9008** encodes the vector to a non-zero binary value identifying 2D (search) Engine_0 **150** as having the matching memory string.

(86) Continuing with the example, responsive to a plurality of memory strings of 2D (search) Engine_0 **150** detecting matches as well as a single memory string of 2D (search) Engine_128K-2 **158** detecting a match, SAs **9007** provides a vector with a bit corresponding to 2D (search) Engine_0 **150** as a one, another bit corresponding to 2D (search) Engine_128K-2 **158** as a one, and all other bits as zeros. In response, Page Buffer, Cache, Priority Encoder **9008** encodes the vector to a non-zero binary value identifying 2D (search) Engine_128K-2 **158** as having the highest-priority matching memory string.

3D (Search) Engine Encoding

(87) Searching, comparing, and/or sorting as performed by a 3D (search) engine is according to a selected feature encoding (for feature storage in memory strings of the 3D (search) engine) and a selected search encoding (for searching performed by the memory strings). The feature and search encodings are selected in conjunction with each other, as matching performed by the memory strings is in the context of the encoding selected for the feature storage.

(88) Example encodings usable in 3D (search) engines for storing features and/or performing searches include expansion encoding, one-hot encoding, low effect encoding (conceptually a variation of expansion encoding that enables reduced pattern and body effects), and reverse expansion encoding.

(89) As a brief introduction, FIG. 1E and FIG. 1F respectively illustrate expansion encoding and one-hot encoding usage examples in a context of 2D (search) Engine_0 **150**, in a similar style as FIG. 1D. Correspondingly, FIG. 2 and FIG. 3 respectively illustrate expansion encoding and one-hot encoding examples.

3D (Search) Engine Expansion Encoding

(90) Turning first to expansion encoding, an N-bit binary value (either a feature or a search) is encoded in a $2^{\lceil \log_2 N \rceil}$ bit field. The encoding produces two contiguous fields of bits. Each of the contiguous fields is a repeat of either a one or a zero. Thus, $2^{\lceil \log_2 N \rceil} - 1$ is encoded as a single leading zero and the remainder all ones. The number of ones is equal to the value being encoded. As a special case, a zero is encoded as all zeros. Thus, the most-significant bit is always zero. Therefore, in some variations, the most-significant bit is not implemented.

(91) FIG. 2 illustrates an example expansion encoding for a 3D (search) engine. There are two columns in a table. The first column, Value 251, is a 6-bit binary value to be encoded according to expansion encoding. The second column, Expansion Encoding 252, is a 64-bit binary expansion encoding of the corresponding binary value from the first column. For readability, the 64-bit values are separated every four bits by an underscore (“_”) character. For brevity, the table omits some values.

(92) FIG. 1E illustrates an expansion encoding usage example of a 2D (search) engine of a 3D (search) engine, specifically 2D (search) Engine_0 **150** of FIG. 1D. Elements of FIG. 1E and FIG. 1D having identical element identifiers are identical elements.

(93) Four example features encoded according to the expansion encoding of FIG. 2 are summarized in a table in the left-hand portion of FIG. 1E. In particular, MemStr0 130 is programmed to store Feature 0 1000E with an expansion encoding of 1 (a single one bit and 63 zero bits). Each feature bit is stored in a series pair of memory devices. The feature expansion encoding is performed by Feature Encoder 9003 of FIG. 1A. MemStr1 131 is programmed to store Feature 1 1001E with an expansion encoding of 4 (four one bits and 60 zero bits). Features 2-509 are programmed to store expansion encodings of various values not specifically discussed. MemStr510 138 is programmed to store Feature 510 1510E with an expansion encoding of 5 (five one bits, four of which are illustrated, and 59 zero bits). MemStr511 139 is programmed to store Feature 511 1511E with an expansion encoding of 63 (63 one bits and a single zero bit).

(94) In the illustrated usage scenario, searches are also encoded according to the expansion encoding of FIG. 2. The search expansion encoding is performed by Search Encoder 9005 of FIG. 1A. A search for the value 1 is encoded on pairs of word lines as a search for a single one bit and 63 zero bits. Assuming the search is in block 0, the pairs of word lines used for the search are WL0_0, WL1_0 . . . WL126_0, and WL127_0. Responsive to the expansion encoding of 63 presented on the word lines of block zero, MemStr0 130 enables a current flow as indicated by String Current 136E. The current flow indicates a match as sensed by SA 143 and presented on Sense Output 144 for further processing.

(95) Responsive to none of the stored features matching any of the corresponding search inputs, the current flow indicates no match, as sensed by SA 143, and presented on Sense Output 144 for further processing.

(96) Expansion encoding is usable for approximate matching between searches and stored features. A best match (e.g., high similarity) has a highest memory string current (e.g., lowest impedance). A worst match (e.g., low similarity) has a lowest memory string current (e.g., highest impedance). A number of bits matched (or mismatched) is determinable by measurement of memory string current and/or impedance.

(97) Example search expansion encoding resolutions include 4 bits implemented using 32 3D-NAND layers, 5 bits using 64 3D-NAND layers, 6 bits using 128 3D-NAND layers, 7 bits using 256 3D-NAND layers, and 8 bits using 512 3D-NAND layers. Thus, resolution increases by one bit as the number of 3D-NAND layers is doubled.

3D (Search) Engine One-Hot Encoding

(98) Turning next to one-hot encoding, an N-bit binary value (either a feature or a search) is encoded in a 2^N field. The encoding produces a single one bit with all other bits zeros. The single one bit is in a bit position that corresponds to the value being encoded.

(99) FIG. 3 illustrates an example one-hot encoding for a 3D (search) engine, as a table. There are two columns in the table. The first column, Value 351, is a 6-bit binary value to be encoded according to one-hot encoding. The second column, One-Hot Encoding 352, is a 64-bit binary one-hot encoding of the corresponding binary value from the first column. For readability, the 64-bit values are separated every four bits by an underscore (“_”) character. For brevity, the table omits some values.

(100) FIG. 1F illustrates a one-hot encoding usage example of a 2D (search) engine of a 3D (search) engine, specifically 2D (search) Engine_0 150 of FIG. 1D. Elements of FIG. 1F and FIG. 1D having identical element identifiers are identical elements.

(101) Four example features encoded according to the one-hot encoding of FIG. 3 are summarized in a table in the left-hand portion of FIG. 1F. In particular, MemStr0 130 is programmed to store Feature 0 1000F with a one-hot encoding of 1 (a single one bit in bit position 1 and 63 zero bits elsewhere). Each feature bit is stored in a series pair of memory devices. The feature one-hot encoding is performed by Feature Encoder 9003 of FIG. 1A. MemStr1 131 is programmed to store Feature 1 1001F with a one-hot encoding of 3 (a single one bit in bit position 3 and 63 zero bits elsewhere). Features 2-509 are programmed to store one-hot encodings of various values not specifically discussed. MemStr510 138 is programmed to store Feature 510 1510F with a one-hot encoding of 4 (a single one bit in bit position 4 and 63 zero bits elsewhere). MemStr511 139 is programmed to store

Feature 511 **1511F** with a one-hot encoding of 63 (a single one bit in bit position 63, the most-significant bit, and 63 zero bits elsewhere).

(102) In the illustrated usage scenario, searches are also encoded according to the one-hot encoding of FIG. 3. The search one-hot encoding is performed by Search Encoder **9005** of FIG. 1A. A search for the value 1 is encoded on pairs of word lines as a search for a single one bit in bit position 1 and 63 zero bits. Assuming the search is in block 0, the pairs of word lines used for the search are WL0_0, WL1_0 . . . WL126_0, and WL127_0. Responsive to the one-hot encoding of 1 presented on the word lines of block zero, MemStr0 **130** enables a current flow as indicated by String Current **136F**. The current flow indicates a match as sensed by SA **143** and presented on Sense Output **144** for further processing.

(103) Responsive to none of the stored features matching any of the corresponding search inputs, the current flow indicates no match, as sensed by SA **143**, and presented on Sense Output **144** for further processing.

3D (Search) Engine Low Effect Encoding

(104) Turning next to low effect encoding, the encoding is a variation on expansion encoding that enables reduced pattern effects and reduced body effect of the floating-gate transistors in memory strings. Conceptually, floating-gate transistors and corresponding word lines are rearranged so that floating-gate transistors programmed at a same threshold voltage tend to be contiguous with each other within a memory string. This is accomplished without actually moving transistors around but by altering the expansion encoding.

(105) FIG. 4A illustrates an example expansion encoding and a corresponding example low effect encoding. For brevity, the examples are with respect to memory strings of 16 floating-gate transistors and correspondingly 16 word lines.

(106) There are three columns arranged in a tabular fashion. The first column, Value **461**, is a binary value to be encoded. The second column, Expansion Encoding **462**, is an 8-bit expansion encoding of the value from the first column. The third column, Low Effect Encoding **463**, is an equivalent 8-bit low effect encoding of the value from the first column.

(107) The table has three sections of rows. The first section, Search Inputs **404**, is a single row representing respective encodings of a same search value of 3. The second section, Stored Features **464**, has nine rows each representing a respective stored feature value according to one of the expansion encoding or the low effect encoding. The third section, Memory Strings **430**, indicates memory strings that progress from left to right respectively starting from a bit line “BL” through a string select line “SSL” through a plurality of word lines “WL . . . WL”, through a ground select line “GSL” and ending at a current source line “CSL”.

(108) The “HL” entries in Expansion Encoding **462** correspond to a data bit of one, and the “LH” entries correspond to a data bit of zero. Each “H” or “L” character corresponds to a threshold voltage of a floating-gate transistor. The “H” entries in Stored Features **464** correspond to a “high” threshold voltage, and the “L” entries correspond to a “low” threshold voltage. The H2 and H1 entries in Search Inputs **404** correspond to two respective word line voltages. The particular search input is illustrated as 3. The underlining in Expansion Encoding **462** and Low Effect Encoding **463** serves to indicate the conceptual rearrangement of the floating-gate transistors and the corresponding word lines.

(109) FIG. 4B illustrates an example expansion encoding with Hamming distances compared to an input of 1. The input of 1 corresponds to a search input of 1 and is illustrated encoded according to expansion encoding as Input **450**. The example encoding is that of FIG. 2. There are three columns in a table. The first column, Value **451**, is a 6-bit binary value to be encoded according to expansion encoding. The second column, Expansion Encoding **452**, is a 64-bit binary expansion encoding of the corresponding binary value from the first column. The third column, HD **453**, is the Hamming distance between the respective expansion encoding and the expansion encoding of search input of 1 (Input **450**). The Hamming distance is a minimum (0) when the input matches the stored value. The Hamming distance is a maximum (62) when the input mismatches nearly all the bits of the stored

value. For readability, the 64-bit values are separated every four bits by an underscore (“_”) character. For brevity, the table omits some values.

(110) FIG. 4C illustrates example threshold voltages and word line voltages that enable comparing of ones, zeros, and don't cares. The left portion of the figure represents memory state “1” cells, as Memory State “1” 471. The right portion of the figure represents memory state “0” cells, as Memory State “0” 470. The figure represents two sets of relationships between threshold voltages and word line voltages with respect to storing features in floating-gate transistors and driving search information on word lines to control gates of the floating-gate transistors. Either set is usable according to usage scenario.

(111) The first set corresponds to a first word line voltage VH1 having a value of V2 473 and a second word line voltage VH2 having a value of V3 474. Thus, with respect to floating-gate transistors programmed to be in Memory State “1” 471, application of either VH1 (having the value V2 473) or application of VH2 (having the value V3 474), results in a high gate overdrive. With respect to floating-gate transistors programmed to be in in Memory State “0” 470, application of VH2 (having the value V3 474) results in a high gate overdrive. However, application of VH1 (having the value V2 473) results in low gate overdrive of those floating-gate transistors.

(112) The second set corresponds to a first word line voltage VL having a value of V1 472 and a second word line voltage VH having a value of V3 474. Thus, with respect to floating-gate transistors programmed to be in Memory State “1” 471, application of VL (having the value of V1 472) results in high gate overdrive. Further, with respect to floating-gate transistors programmed to be in Memory State “0” 470, application of VL (having the value of V1 472) results in no gate overdrive. With respect to floating-gate transistors programmed to be in either Memory State “1” 471 or Memory State “0” 470, application of VH having the value V3 474 results in high gate overdrive.

(113) FIG. 4D illustrates icons relating high and low gate overdrive to low and high floating-gate transistor and cell impedances, conceptually corresponding respectively to high and low cell current capabilities. High Gate Overdrive 483 represents, by a relatively thick arrow, high gate overdrive. Low Gate Overdrive 484 represents, by a relatively thin arrow, low gate overdrive. High Cell Current 485 indicates, by two relatively thick arrows, relatively high cell current capability (e.g., relatively low impedance). Low Cell Current 486 and Low Cell Current 487 indicate, by one relatively thick arrow and one relatively thin arrow, relatively low cell current capability (e.g., relatively high impedance).

(114) FIG. 4E illustrates floating-gate transistor current capability (e.g., impedance) as a function of search information versus stored data. Stored Data Values 499 has four columns, Stored 0 490, Store 1 491, Store Don't Care 492, and Store Invalid 493. Input Search Information 489 has three rows, Search 0 480, Search 1 481, and Wildcard 482. Each cell in the table represents impedance of each floating-gate transistor, in accordance with the icons illustrated in FIG. 4D, as well as effective impedance of the pair of floating-gate transistors depicted in each cell, also in accordance with the icons illustrated in FIG. 4D. Cells 4900, 4902, 4911, 4912, 4920, 4921, 4922, and 4932 represent gate overdrive of both floating-gate transistors in a pair, resulting in a relatively lower impedance. Cells 4901, 4910, 4930, and 4931 represent gate overdrive of only one floating-gate transistor in a pair, resulting in a relatively higher impedance. The relatively lower impedance represents a match between search and data, and the relatively higher impedance represents otherwise.

(115) As a specific example, consider Cell 4900 in a usage scenario of the first set of relationships between threshold voltages and word line voltages VH1 (having the value V2 473) and VH2 (having the value V3 474). Vy is equal to VH2 and Vx is equal to VH1. Thus, the upper floating-gate transistor, controlled by Vy (equal to VH2) has high gate overdrive, and thus a high current capability corresponding to a low impedance. The lower floating-gate transistor, controlled by Vx (equal to VH1) also has high gate overdrive, and thus a high current capability corresponding to a low impedance. Therefore, overall, Cell 4900 has a high current capability corresponding to a low impedance.

(116) As another specific example, consider Cell 4900 in a usage scenario of the second set of

relationships between threshold voltages and word line voltages VL (having the value of V1 **472**) and a second word line voltage VH (having the value V3 **474**). Vy is equal to VH and Vx is equal to VL. Thus, the upper floating-gate transistor, controlled by Vy (equal to VH) has high gate overdrive, and thus a high current capability corresponding to a low impedance. The lower floating-gate transistor, controlled by Vx (equal to VL) also has high gate overdrive, and thus a high current capability corresponding to a low impedance. Therefore, overall, Cell **4900** has a high current capability corresponding to a low impedance.

3D (Search) Engine Reverse Expansion Encoding

(117) Turning next to reverse expansion encoding, the encoding is conceptually a logical inversion of expansion encoding, but applied only to search inputs, rather than features. Reverse expansion encoding is applicable to approximate matching.

(118) FIG. 5A illustrates an example reverse expansion encoding for search inputs of a 3D (search) engine, in conjunction with an example (non-reverse) expansion encoding for stored features, as a table.

(119) The table is organized in three sections. An upper section describes reverse expansion encoding, as applied to search values to generate search inputs for application to word lines. A middle section describes an example input as reverse expansion encoded. A lower section describes stored value expansion encodings and Hamming distances with respect to the example input.

(120) The first column in the upper section, Search Value **555**, is a 6-bit binary search value to be encoded according to reverse expansion encoding. The second column, Reverse Expansion Encoding **554**, is a 64-bit binary reverse expansion encoding of the corresponding binary value from the first column. Note that similar to expansion encoding, in reverse expansion encoding to a 64-bit binary value, the most-significant bit is always one. Therefore, in some variations, the most-significant bit is not implemented.

(121) The middle section is a reverse expansion encoding of the search value one, as Search Value Example **550**.

(122) The first column in the lower section, Stored Value **551**, is a 6-bit binary feature to be encoded according to expansion encoding for storage in memory strings. The second column, Expansion Encoding **552**, is a 64-bit binary expansion encoding of the corresponding binary value from the first column. The third column, HD **553**, is the Hamming distance between a stored value having the respective expansion encoding and the search input of 1 (Search Value Example **550**) in the reverse expansion encoding. Recall that in a 64-bit binary expansion encoding the most-significant bit is always zero and that in a 64-bit binary reverse expansion encoding the most-significant bit is always one. Therefore, comparing a particular value encoded according to the 64-bit binary expansion encoding to the particular value encoded according to the 64-bit binary reverse expansion encoding, always mismatches in the most-significant bit. Therefore, HD **553** ignores the most-significant bit. The Hamming distance is a minimum (1) when the stored value is closest to the search input in the reverse expansion encoding (when ignoring the most-significant bit). The Hamming distance is a maximum (64) when the stored value is the furthest from the search input in the reverse expansion encoding (when ignoring the most-significant bit).

(123) For readability, 64-bit values in the table are separated every four bits by an underscore (“_”) character. For brevity, the table omits some values.

(124) FIG. 5B graphically illustrates, for reverse expansion encoding, example Hamming distances (HDs **599**) as a function of stored data values (Stored Data Value **590**) and search input values (Search Input Value **506**).

(125) FIG. 6 graphically illustrates, for expansion encoding, example Hamming distances (HDs **699**) as a function of stored data values (Stored Data Value **690**) and search input values (Search Input Value **606**). FIG. 6, compared against FIG. 5B, depicts how reverse expansion encoding is “reverse” with respect to expansion encoding.

(126) With respect to FIG. 5B and FIG. 6, relatively lower HDs are depicted in relatively lighter shadings compared to relatively higher HDs that are depicted in relatively darker shadings. The

lowest HD is 0 (indicating equality) and is depicted in the lightest shading. The highest HD is 63 (indicating 63 bits mismatch) and is depicted in the darkest shading.

(127) In FIG. 5B (reverse expansion encoding) HDs are 0 along the diagonal. In contrast, in FIG. 6 (expansion encoding) HDs along the diagonal are 63. Further, in FIG. 5B (reverse expansion encoding) HDs are 63 at the lower-left and upper right corners. In contrast, in FIG. 6 (expansion encoding) HDs in the lower-left and upper-right corners are 0. Thus, the HDs of FIG. 5B are “reverse” of those of FIG. 6.

(128) Reverse expansion encoding is usable for approximate matching between searches and stored features. A best match (e.g., high similarity) has a lowest memory string current (e.g., highest impedance). A worst match (e.g., low similarity) has a highest memory string current (e.g., lowest impedance). A number of bits matched (or mismatched) is determinable by measurement of memory string current and/or impedance.

3D (Search) Engine One-Hot Encoding Stored and Input Ranges

(129) FIG. 7A illustrates an example of one-hot encoding with stored ranges, as a multi-part table. The table is organized in two sections. An upper section describes an example input (the value 2) as one-hot encoded as Input 750. A lower section describes one-hot encoded stored ranges and match information with respect to the example input.

(130) The first column in the lower section, Value Range 751, is a list of respective ranges of 6-bit features to be encoded and stored as features to match search inputs against. E.g., 0-2 specifies a stored feature to match a search input from 0-2 (0, 1, or 2). The second column, One-Hot Encoding with Ranges 752, is a 64-bit binary one-hot encoding for the range from the first column. The character “X” indicates the corresponding pair of floating-gate devices are configured to match either a search input of one or zero, e.g., a don't care. The third column, Match? 753, is whether the example input of 2 (One-Hot Encoding with Ranges 752) matches the corresponding stored range feature. The example input of 2 matches the first three one-hot encoded ranges in the table. Note that a one-hot encoding without ranges (e.g., Expansion Encoding 352 of FIG. 3) only matches the example input of 2 with a stored feature value of 2.

(131) For readability, 64-bit values in the table are separated every four bits by an underscore (“_”) character. For brevity, the table omits some values.

(132) FIG. 7B illustrates a one-hot encoding on input range example, as a multi-part table. The table is organized in two sections. An upper section describes an example input range (0-4) as one-hot encoded as One-Hot Encoding on Input Range Example 760. The character “X” indicates the corresponding search input is a Wildcard to match any corresponding stored feature bit. A lower section describes one-hot encoded stored values and match information with respect to the input range example.

(133) The first column in the lower section, Value 761, is a 6-bit binary value to be encoded according to one-hot encoding. The second column, One-Hot Encoding 752, is a 64-bit binary one-hot encoding of the corresponding binary value from the first column. The third column, Match? 763, is whether the example input range of 0-4 matches the corresponding stored feature. The example input range of 0-4 matches the first three one-hot encoded features in the table.

(134) For readability, the 64-bit values are separated every four bits by an underscore (“_”) character. For brevity, the table omits some values.

(135) One-hot encoding is usable for exact matching between searches and stored features. One-hot encoding with stored ranges is usable for approximately matching searches. Programming a stored feature with more don't cares increases the range of search inputs that the stored feature matches. One-hot encoding of input ranges is usable for approximately matching searches. Altering which bits are Wildcards moves the range of search inputs that match. For example, a stored feature of XX1XX000 . . . 00 matches search inputs of 0 to 4, and a stored feature of 0XX1XX000 . . . 00 matches search inputs of 1 to 5.

3D (Search) Engine Operation

(136) FIG. 8 illustrates an example operating flow for a 3D (search) engine (such as 3D Engine 9000

of FIG. 1A), as Engine Flow **800**. Flow begins at Select Feature and Search Encodings **801**. In some usage scenarios, identical encodings are used for features and searches. In some usage scenarios, e.g., reverse expansion encoding, different encodings are used for features versus searches. Flow then proceeds to Program Features **802**. The features to be searched against are programmed according to the selected feature encoding. Flow then proceeds to Encode Search **803**. The search is encoded according to the selected search encoding. In some applications, a search is performed based on one or more parameters optionally including a mask and/or range information. Flow then proceeds to Drive Word Lines **804**. The word lines are driven according to the search encoding. Flow then proceeds to Sense Bit Lines **805**. The bit line sensing is performed by sense amplifiers. Flow then proceeds to Priority Encode Matches **806**. Match indications, as provided by the sense amplifiers, are processed to determine prioritized match information (e.g., as performed by Page Buffer, Cache, Priority Encoder **9008** of FIG. 1A). Flow then proceeds to Provide Results **807**. The results are provided, for example, to a host agent (e.g., as Output Matching Result(s) **9009** of FIG. 1A). Flow then optionally proceeds back to encode a next search (Encode Search **803**).

3D (Search) Engine Device and Appliance

(137) FIG. 9 illustrates an example 3D engine appliance comprising an example 3D engine device, respectively as 3D Engine Appliance **940** and 3D Engine Device **950**.

(138) 3D Engine Appliance **940** comprises one or more instances of 3D engine devices, for clarity illustrated as a single device, 3D Engine Device **950**, as well as Appliance Interface **941**.

(139) 3D Engine Device **950** comprises 3D Engine **9000** (e.g., as illustrated and described with respect to FIGS. 1A-1C). 3D Engine Device **950** further comprises Controller **951**, Engine Interface **953**, and Analog Source **954**. Conceptually, elements of 3D Engine Device **950** and/or 3D Engine Appliance **940** (other than 3D Engine **9000**) serve to provide 3D Engine **9000** with Features **9001** and Searches **9002** from a host, and to return to the host Result(s) **9099**.

(140) Controller **951** is enabled to provide overall control of operations of and between various hardware circuitry blocks of 3D Engine Device **950**. For example, Controller **951** coordinates operation of Engine Interface **953** with 3D Engine **9000**. For another example, Controller **951** coordinates operation of Analog Source **954** with respect to configuration information. For another example, Controller **951** optionally coordinates feature and/or search encoding as well as feature programming as directed via Engine Interface **953** and as applicable to 3D Engine **9000**. For other examples, Controller **951**, in various configurations, is enabled to perform operations relating to any one or more of operations **801-807** of Fig.8.

(141) Engine Interface **953** is enabled to interface an agent external to 3D Engine Device **950** (such as a host via Appliance Interface **941** and Host/Bus Coupling **942**) to use 3D Engine **9000**. Example uses are to receive commands and data, as well as to return status relating to feature and/or search encoding, feature programming, and/or search requesting.

(142) Analog Source **954** is enabled to generate and/or provide one or more analog outputs to, e.g., memory arrays, bit line circuitry, sense amplifier circuitry, and/or word line circuitry of the 3D (search) engine memory arrays. The analog outputs comprise zero or more voltage and/or current sources (such as reference sources), for instance as generated by one or more bias circuits.

(143) In operation, a host provides, e.g., commands for selection of feature and/or search encoding via Host/Bus Coupling **942** and Appliance Interface **941**. Appliance Interface **941** then provides the commands to Engine Interface **953** and the commands are processed by 3D Engine **9000** as directed by Controller **951**. Examples of other similarly processed commands include feature programming according to the selected feature encoding, and search performing according to the selected search encoding. Status or processed commands as well as results of searches are returned to the host via Host/Bus Coupling **942** and Appliance Interface **941** as provided by Controller **951** and/or 3D Engine **9000** via Engine Interface **953**. In some variations of 3D Engine Device **950**, Controller **951** is enabled to operate one or more 2D (search) engines of 3D Engine **9000** concurrently.

(144) In operation, a host external to 3D Engine Appliance **940** provides feature information for 3D Engine Device **950** to program into 3D Engine **9000**. Subsequently, the host provides one or more

search parameters to 3D Engine Appliance **940**. 3D Engine Appliance **940**, via Appliance Interface **941**,

(145) Although FIG. **9** illustrates 3D Engine **9000** used in 3D Engine Device **950** in turn used in 3D Engine Appliance **940**, other usage scenarios are possible. For example, various elements (e.g., integrated circuits, one or more die of a system-on-a-chip, and/or packaged die) comprise one or more 3D engine elements identical to, substantially similar to, or based on 3D Engine **9000**.

(146) Other partitionings of elements, coupling between elements, and capabilities and/or capacities of elements illustrated in the figure are contemplated, as well as additional elements, according to usage requirements.

3D (Search) Engine System

(147) FIG. **10** illustrates an example hardware system having a 3D (search) Engine, as 3D Engine System **1000**. The system comprises hardware blocks CPU **1010**, GPU **1020**, RAM **1030**, 3D Engine Appliance **940** (of FIG. **9**), SSD **1050**, HDD **1060**, and I/O **1070** that are coupled by hardware block Bus(es)/Interface(s) **1080**.

(148) CPU **1010** comprises one or more processing units, such as any combination of hardware units enabled to execute programmed instructions, microprocessors, signal processors, AI processors, and the like.

(149) GPU **1020** comprises one or more processing units, such as any combination of units enabled to accelerate processing for processing that is subject to relatively highly parallel processing, such as graphics processing, signal processing, and/or AI processing.

(150) RAM **1030** comprises one or more storage elements for storage of instructions and/or data in greater quantities than storage internal to CPU **1010** and/or GPU **1020**. RAM **1030** is implementable, e.g., via volatile memory elements, such as DRAMs and/or SRAMs.

(151) SSD **1050** comprises one or more storage elements, such as flash-based storage elements for storage of instructions and/or data optionally accessible with reduced latency compared to HDD **1060**.

(152) HDD **1060** comprises one or more storage elements, such as rotation-based magnetic and/or optical non-volatile storage elements (e.g., disks) for storage of instructions and/or data optionally in greater quantities than SSD **1050** is enabled to store.

(153) I/O **1070** comprises elements to interface any combination of CPU **1010**, GPU **1020**, RAM **1030**, 3D Engine Appliance **940**, SSD **1050**, and/or HDD **1060** to elements external to 3D Engine System **1000**. Example external elements include mass storage devices, local and wide-area networks (such as the Internet), human interface components (such as keyboards, mice, and/or monitors), and other elements providing capabilities to extend and/or augment capabilities not otherwise provided by 3D Engine System **1000**. I/O **1070** variously comprises one or more serial and/or parallel communication channels as well as optional protocol conversion and/or adaptation capabilities to facilitate communication between the elements coupled to it.

(154) Bus(es)/Interface(s) **1080** enables communication between the elements coupled to it (e.g., CPU **1010**, GPU **1020**, RAM **1030**, 3D Engine Appliance **940**, SSD **1050**, HDD **1060**, and/or I/O **1070**). Bus(es)/Interface(s) **1080** variously comprises one or more serial and/or parallel communication channels as well as optional protocol conversion and/or adaptation capabilities to facilitate communication between the elements coupled to it.

(155) Other partitionings of elements, coupling between elements, and capabilities and/or capacities of elements illustrated in the figure are contemplated, as well as additional elements, according to usage requirements.

(156) In some circumstances, 3D Engine Appliance **940** enables reductions in processing load (e.g., on CPU **1010** and/or GPU **1020**) and/or reductions in traffic on Bus(es)/Interface(s) **1080** by performing processing locally.

(157) For example, using one-hot encoding (e.g., for stored features and/or for search input ranges) to enable range matching, searches among large databases are accomplished entirely within 3D Engine Appliance **940** once features are programmed. For another example, using reverse expansion

encoding to enable approximate matching, comparisons among large AI datasets are accomplished entirely within 3D Engine Appliance **940** once features are programmed. For another example, using expansion and/or one-hot encoding, sorting large datasets is accomplished entirely within 3D Engine Appliance **940** once features are programmed.

3D (Search) Engine Additional Information

(158) Example memory technologies applicable to memory arrays of 3D (search) engines as disclosed herein include floating-gate, split-gate, SONOS, floating dot, DRAM, DRAM-like (e.g., 2T0C), FeFET, and any memory technology compatible with search via word lines and bit lines. Exemplary SONOS memory technology (sometimes referred to as charge trap memory) uses an insulating layer (e.g., of silicon nitride) with traps to capture and retain charge as injected from a channel. Exemplary floating dot memory technology conceptually replaces a floating gate with a floating silicon nanodot or embeds floating silicon nanodots in a polysilicon gate. Exemplary 2T0C memory technology uses parasitic capacitance of a read transistor to store charge rather than an explicit storage capacitor. Exemplary FeFET memory technology uses permanent electrical field polarization of ferroelectric material embedded between a gate and a source-gate conduction region to store information. Example memory structures applicable to memory arrays of 3D (search) engines include 2D structures (e.g., 2D flash structures) and 3D structures (e.g., 3D flash structures). Example array architectures applicable to memory arrays of 3D (search) engines include NOR/OR-type array architectures and AND/NAND-type array architectures.

(159) It is understood that the foregoing disclosure presents implementations, variations, embodiments, and examples in an intended illustrative sense rather than in a limiting sense. It is contemplated that modifications and combinations are discernible that will be within the spirit of the disclosure and the scope of the following claims. What is claimed is:

Claims

1. A computing system comprising: a plurality of memories each comprising: a respective match line coupled to a respective sense amplifier, a respective source line, and a respective plurality of memory strings each coupled to (a) the respective match line and (b) the respective source line, and each memory string comprising a respective plurality of series-connected floating-gate transistors each having and being responsive to (c) a respective configured state and (d) a respective control input; and a plurality of word lines each coupled to a respective one of the respective control inputs of each of the series-connected floating-gate transistors; and a search encoder enabled to receive a search and drive the word lines according to a search encoding; wherein (e) each memory string is enabled to couple the respective match and source lines that the respective memory string is coupled to via an a match-found impedance responsive to the configured states of the memory string exactly matching the control inputs of the series-connected floating-gate transistors of the memory string, and (f) each sense amplifier is enabled to generate a respective indication of whether the match line it is coupled to is coupled to the source line via the match-found impedance; and wherein (g) each memory string is further enabled to couple the respective match and source lines that the memory string is coupled to via a no-match-found impedance responsive to the configured states of the memory string not exactly matching the control inputs of the series-connected floating-gate transistors of the memory string, and (h) each sense amplifier is further enabled to generate a respective indication of whether the match line it is coupled to is coupled to the source line via the no-match-found impedance.
2. The computing system of claim 1, wherein the word lines are operated in pairs according to the search encoding and the configured states are managed in pairs according to a feature encoding.
3. The computing system of claim 1, wherein each of the configured states corresponds to a respective threshold voltage.
4. The computing system of claim 3, wherein each of the respective control inputs is coupled to a respective gate of one more of the series-connected floating-gate transistors.
5. The computing system of claim 3, wherein the threshold voltages are settable via one or more

programming operations.

6. The computing system of claim 1, wherein each of the memory strings comprises a respective memory string enable device responsive to a respective memory string enable control input to selectively enable or disable the respective memory string independently of the word lines.

7. A computing method comprising: receiving one or more search parameters; encoding the search parameters according to a particular one of a plurality of search encodings to generate pairs of word line voltages, the particular search encoding enabling a search for a match with a feature stored in a pair of floating-gate transistors according to a particular one of a plurality of feature encodings, and wherein the particular search encoding and the particular feature encoding enable representation of a one, a zero, and either a one or a zero; driving pairs of word lines of a plurality of series-connected instances of the pair of floating-gate transistors according to the generated word line voltages, the series-connected instances organized as respective pluralities of memory strings operable in parallel to affect a plurality of shared match lines, each shared match line coupled to two or more of the memory strings, and each pair of word lines coupled to at least two of the instances of the pair of floating-gate transistors; sensing voltage changes on the shared match lines to determine which of the shared match lines have a voltage change greater than a predetermined sense threshold; and providing results of the determining as indications of relative matching between the search parameters and the stored features.

8. The computing method of claim 7, wherein at least one of the plurality of search encodings or at least one of the plurality of feature encodings is in accordance with at least one of an expansion encoding and a one-hot encoding.

9. The computing method of claim 7, wherein at least one of the plurality of search encodings or at least one of the plurality of feature encodings is in accordance with a reverse expansion encoding.

10. The computing method of claim 7, wherein at least one of the stored features corresponds to a range stored using a one-hot encoding.

11. The computing method of claim 7, wherein at least one of the pairs of word line voltages is in accordance with a range encoded as a one-hot encoding.

12. The computing method of claim 7, wherein at least one of the plurality of search encodings or at least one of the plurality of feature encodings is in accordance with a low effect encoding.

13. A computing system comprising: interface circuitry to enable a host agent to provide a search and to receive one or more results of the search; a plurality of blocks of memory strings, each block comprising a respective plurality of memory strings each receiving a same set of word lines having voltages determinable responsive to the search; a search encoder enabled to receive the search and drive the word lines according to a search encoding; a plurality of sense amplifiers each coupled to a corresponding one of the memory strings from each of the blocks; and priority encoder circuitry enabled to receive match indications from the sense amplifiers; wherein the sense amplifiers are enabled to determine, with respect to matches between (1) feature information stored in the memory strings coupled to the sense amplifier and (2) the word lines received by the memory strings coupled to the sense amplifier, that (3) there are no matches and (4) there is at least one match; and wherein the priority encoder circuitry, based on the determinations of the sense amplifiers, is enabled to indicate at least a portion of the results as a highest priority match of any matches identified by the sense amplifiers.

14. The computing system of claim 13, further comprising the host agent.

15. The computing system of claim 13, wherein the priority encoder circuitry, based on the determinations of the sense amplifiers, is enabled to indicate at least a portion of the results as a second highest priority match of any matches identified by the sense amplifiers.

16. The computing system of claim 13, wherein the feature information is stored in the memory strings by configuring threshold voltages of floating-gate transistors of the memory strings.

17. The computing system of claim 13, wherein two or more of the blocks are operable concurrently.

18. The computing system of claim 13, wherein a feature encoding is used to encode at least some of the feature information.

19. The computing system of claim 18, wherein the feature encoding is at least one of an expansion encoding, a reverse expansion encoding, a one-hot encoding, and a low effect encoding.

20. The computing system of claim 13, wherein the computing system is part of a system-on-a-chip (SOC).
